TaxonVision-MLOps

End-to-End Architectural and Operational Strategy Report

Systems Engineering

Benjamin Förster

October 13, 2026

Abstract

TaxonVision-MLOps establishes an automated, production-grade Machine Learning Operations platform for fine-grained biological species identification. Transitioning away from ad-hoc, notebook-bound model development, this architecture utilises a unified open-source toolchain (DagsHub, DVC, MLflow, Optuna) mapped to a hybrid CI/CD topology: GitHub Actions executes the lightweight quality gates, whilst a self-hosted Jenkins controller schedules all GPU-bound work as ephemeral Kubernetes pods on a single bare-metal host. Model training, hyperparameter search and ONNX compilation are performed exclusively on this remote execution plane; the developer workstation serves solely for authoring and orchestration. In order to comply strictly with ethical and regulatory standards, the system firmly enforces epistemic safety via Split Conformal Prediction and Energy-Based Out-of-Distribution (OOD) detection, thereby ensuring mathematical coverage guarantees in production. This document carefully outlines the end-to-end pipeline, testing strategies, and core architectural trade-offs driving the system's overall design, and it records the verified implementation status against the phased delivery roadmap.

Contents

Abstract **i**

Preface **1**

1 Architectural Context and Operational Maturity **2**
1.1 Data Topography and Open-Access Mandates . . . 2
1.2 The Operational Bottleneck in Biodiversity Monitoring . . . 2
1.3 The TaxonVision Operational Maturity Framework . . . 3
1.4 Remote-Only Execution Principle . . . 3
1.5 Normative Architecture and Implementation Status . . . 4

2 Physical Infrastructure and Data Topology **6**
2.1 Heterogeneous Silicon and Physical Determinism . . . 6
2.2 Verified Software Baseline of the Host . . . 7
2.3 GPU Partitioning and Memory Budget . . . 7
2.4 Access Topology and Edge Security . . . 7
2.5 Cryptographic Storage Economics . . . 7
2.5.1 Circumventing the Multi-Terabyte Storage Trap . . . 8
2.5.2 Bounded Exemplar Storage Mathematics . . . 8
2.5.3 External Remote Fallback Topology . . . 8

3 Open-Source MLOps Toolchain **9**
3.1 Content-Addressable Storage and Cryptographic Versioning . . . 9
3.2 Unified Management Plane . . . 9
3.3 Bayesian Hyperparameter Optimisation and Telemetry . . . 10
3.3.1 Intelligent Search Mechanics . . . 10
3.3.2 Search Space and Optimiser . . . 11
3.3.3 MLflow Interlock Architecture . . . 11
3.4 Training Dynamics and Convergence Control . . . 11

4 Dual Pipeline Architecture **13**
4.1 Asymmetric Iteration and Execution Decoupling . . . 13
4.2 Automated CI/CD Orchestration & Registry Promotion . . . 13
4.3 Automated Training on the Main Branch . . . 15
4.4 Convergence at the Serving Boundary . . . 15
4.5 Low-Latency Interpretability via Forward-Hooked CAM . . . 15

5 Vision RAG & Multimodal Knowledge Graph **17**
5.1 Ingestion Engine & Reference Exemplar Catalogue Curation . . . 17
5.2 Visual Entity Linking: Parent-Child Chunking . . . 18
5.3 Vector Database: Single-Stage Filtered HNSW . . . 18
5.4 Conformal-Driven Lazy Execution . . . 19
5.5 Multi-Modal Re-Ranking via Reciprocal Rank Fusion . . . 20
5.6 Multimodal Context Synthesis & VLM Diagnostics . . . 20

5.7 Operationalizing Taxonomic Mutations in the MMKG 21
5.8 End-to-End Latency & Resource Allocation 21

6 Epistemic Safety & Active Learning 23
6.1 Energy-Based Out-of-Distribution Detection 23
6.2 Spatial-Temporal Constraint Masking 24
6.3 Distribution-Free Uncertainty via Split Conformal Prediction 25
6.3.1 Calibration Protocol and Quantile Estimation 25
6.3.2 Prediction Set Construction & Marginal Coverage Guarantee 25
6.4 Conformal Triage Routing & Active Learning Feedback 26
6.4.1 Active Learning Query Strategies 26

7 Observability, Testing, Deployment 28
7.1 Containerisation and Multi-Stage Runtimes 28
7.1.1 Multi-Stage Image Compilation 28
7.1.2 Kernel-Level Process Isolation 29
7.2 K8s Cluster Orchestration and Workload Routing 29
7.2.1 Asymmetric Pod Architecture and Ingress Routing 29
7.2.2 Self-Healing and Health Probing 29
7.2.3 Horizontal Pod Autoscaling on Custom Telemetry 30
7.3 Telemetry, Metric Exporters, Concept Drift Monitoring 30
7.3.1 Prometheus Telemetry Instrumentation 30
7.3.2 Two-Tier Hybrid Feature Drift Architecture 31
7.3.3 Automated Alerting and Continuous Retraining Triggers 32
7.4 Multi-Tier Verification and Testing Strategy 32
7.4.1 Tier 1: Upstream API Mocking 32
7.4.2 Tier 2: Object Storage and Pipeline Mocking 32
7.4.3 Tier 3: Mathematical Invariant Testing 33
7.4.4 Tier 4: Statistical Metric Selection and Anomaly Verification 33

8 Hardware Optimisation and Memory Mathematics 35
8.1 Dynamic Quantisation and Tensor Compression 35
8.2 Marginal Coverage Guarantees in Conformal Prediction 36
8.3 Memory Complexity of Hierarchical Vector Graphs 36

9 Conclusion and Operational Synthesis 38
9.1 Phased Delivery Roadmap and Implementation Status 38

Preface

With all due respect to the provided curriculum, we must be brutally honest: the reference architecture presented to us in the syllabus is fundamentally flawed for high-throughput computer vision. It attempts to force stochastic, high-dimensional image streams into the rigid, tabular paradigms of MLOps 1.0. To adopt the suggested diagrams wholesale would mean intentionally engineering a system destined to fail under the physical realities of production.

We must critically examine the pedagogical crutches offered to us. The roadmap suggests triggering heavy, GPU-bound training loops via a synchronous HTTP `/train_evaluate` endpoint on the web server. From a systems engineering perspective, this is a catastrophic anti-pattern; it blocks the event loop, starves incoming prediction requests, and guarantees timeout errors under load. Furthermore, the serving diagram proposes querying MLflow over the network to dynamically fetch the production model during live `/predict` inference. This naive approach introduces severe network latency that would instantly destroy any strict Service Level Agreement (SLA). The curriculum also advises loading our datasets into a standard relational database – a methodology that blindly steps into the "50 TB trap" when confronted with the vast reality of global ecological imagery. Finally, it recommends tabular drift detection tools and metric gates based on AUROC. We must clearly state that AUROC is mathematically deceptive and actively dangerous when evaluating extreme power-law distributions in biological data; we instead strictly mandate PR-AUC and AUPIMO to honestly evaluate real-world efficacy.

We simply cannot afford to build on stabiliser wheels. Therefore, the TaxonVision-MLOps architecture completely discards these half-measures in favour of genuine engineering rigour.

Rather than overloading a monolithic web server, this report outlines a strictly decoupled dual-pipeline architecture. We bypass the database bloat entirely by streaming raw TAR archives directly from AWS Open Data into memory via zero-copy Apache Arrow buffers, whilst relying on lightweight DVC Parquet manifests for cryptographic immutability. To guarantee our sub-25 ms latency SLA, we completely reject dynamic network loading; instead, we deploy pre-loaded, statically calibrated INT8 ONNX models that compress the memory footprint by 75% and leverage hardware vector instructions.

Most importantly, we do not rely on uncalibrated, overconfident softmax scores. The system incorporated herein establishes an uncompromising standard of epistemic safety. We filter anomalies natively in the feature space using Energy-Based Out-of-Distribution (OOD) detection. We reject deceptive tabular drift monitors in favour of a two-tier hybrid drift detection strategy: real-time streaming telemetry evaluates the linear-time Wasserstein-1 distance across scalar feature norm projections, whilst offline continuous integration gates enforce full-manifold Maximum Mean Discrepancy (MMD) across the high-dimensional latent space. Crucially, we enforce Split Conformal Prediction, transforming raw probabilities into dynamic prediction sets backed by a distribution-free 95% marginal coverage guarantee ($1 - \alpha$). If the conformal gate detects ambiguity, the system triggers a Single-Stage Filtered HNSW vector traversal, utilising multimodal Vision RAG to synthesise a grounded differential diagnosis.

This report documents a Level 4 MLOps ecosystem. It is not a classroom exercise, but a mission-critical blueprint for global biodiversity monitoring.

Chapter 1

Architectural Context and Operational Maturity

The transition from localised machine learning prototypes to global ecological monitoring platforms requires us to fundamentally rethink how our systems handle scale, uncertainty, and regulatory compliance. In order to achieve this, this chapter establishes the baseline operational context for the TaxonVision-MLOps architecture. We first define the scale of citizen-science data streams, then identify the failure modes of manual data science workflows, and finally formalise the phased Machine Learning Operations (MLOps) maturity framework which is required to sustain a robust production environment.

1.1 Data Topography and Open-Access Mandates

Global biodiversity portals such as iNaturalist and the Global Biodiversity Information Facility (GBIF) aggregate continuous streams of species observations worldwide. Combined, these ecosystems host an estimated 270 million raw observations (as of public registry reporting in 2024–2026), of which roughly 170 million have achieved "research grade" status through community validation. Processing this high-volume stream requires strict cryptographic and legal compliance safeguards right at the ingestion edge.

In order to enforce open-access mandates, the ingestion engine inspects incoming observation metadata headers prior to staging objects locally. A cryptographic licence filtering module evaluates record metadata against allowed open-access definitions, thereby enforcing strict inclusion filters for Creative Commons Zero (CC0), Creative Commons Attribution (CC-BY), and Creative Commons Attribution-NonCommercial (CC-BY-NC). Any incoming observation tagged with restrictive or commercial licences (for example, CC-BY-ND or CC-BY-SA restricted variants) is explicitly dropped at the network boundary. Furthermore, to comply with GBIF and iNaturalist scientific lineage requirements, an automated attribution preservation mechanism extracts observation GUIDs, contributor ORCIDs, and original licence headers, serialising them alongside downstream feature vectors in order to maintain absolute lineage traceability.

1.2 The Operational Bottleneck in Biodiversity Monitoring

Fine-grained biological species identification at a global scale represents a formidable computational challenge. Citizen-science platforms generate continuous, high-volume streams of unstructured visual observations across highly variable environments. Historically, translating these raw field photographs into verified ecological intelligence has been severely constrained by fragile, ad-hoc machine learning engineering.

When predictive models are trained in isolated Jupyter notebooks – a state defined as Level 0 (Manual Process) maturity – they already suffer from non-reproducible software environments, untracked parameter drift, and silent degradation under natural distribution shifts. In an operational setting, these isolated scripts break down entirely during the handoff between data science and deployment teams. This renders the system incapable of sustaining continuous updates or scaling to support thousands of distinct taxa.

1.3 The TaxonVision Operational Maturity Framework

To process citizen-science visual streams reliably, the TaxonVision-MLOps architecture enforces a structured paradigm shift. Rather than relying solely on traditional MLOps reference architectures, we adopt Google's foundational 3-level MLOps spectrum (Levels 0–2: Manual, Automated Pipeline, and CI/CD) and extend it into a tailored 5-Tier Operational Maturity Framework. By introducing Level 3 (Feedback & Active Learning) and Level 4 (Epistemic Safety), the platform bridges standard deployment automation with mathematical coverage guarantees required for ecological monitoring.

Table 1.1: TaxonVision-MLOps 5-Tier Operational Maturity Framework (Adapted from Google MLOps)

Maturity Level	**Infrastructure Configuration**	**Operational Focus**
0: Manual (Google Baseline)	Isolated interactive notebooks; manual state handoffs.	Localized algorithm prototyping.
1: Pipeline (Google Baseline)	Automated DAGs; basic data validation.	Continuous training baseline.
2: CI/CD (Google Baseline)	GitHub Actions & Jenkins; ONNX graph export.	Rapid, repeatable code & model releases.
3: Feedback (TaxonVision Extension)	Telemetry scraping; Active Learning triage queues.	Real-time drift detection & data loops.
4: Safety (TaxonVision Extension)	Split Conformal Prediction; Energy-Based OOD.	Distribution-free coverage guarantees $(1 - \alpha)$.

The comprehensive architecture diagram presented on the following page serves as the visual synthesis of this maturity framework. It maps the end-to-end TaxonVision-MLOps deployment strategy, detailing the physical separation between the offline classification engine and the continuous taxonomic Multimodal Knowledge Graph (MMKG). Furthermore, it illustrates the critical convergence of these dual pipelines at the inference serving boundary, where epistemic safety gates and active learning feedback loops guarantee sustained operational reliability.

1.4 Remote-Only Execution Principle

A further design decision concerns the location at which computation takes place. The developer workstation is deliberately not provisioned for model training. It serves as an authoring, linting and orchestration plane, whereas every training run, hyperparameter search and ONNX compilation is executed on the bare-metal host described in Chapter 2. These jobs are dispatched by the CI/CD controller only after all quality gates have been passed. This principle has three consequences:

- **Reproducibility:** Each run starts from an ephemeral container and a frozen dependency lock file, so that the execution environment cannot drift between runs.

- **Auditability:** The cryptographic dataset hash recorded by DVC is logged to the experiment tracker together with the metrics of every run, which links each model version to the exact data state it was trained on.
- **Resource isolation:** Long-running GPU workloads cannot degrade the interactive responsiveness of the development machine, and they are scheduled alongside the serving workload under an explicit memory budget (Section 2.3).

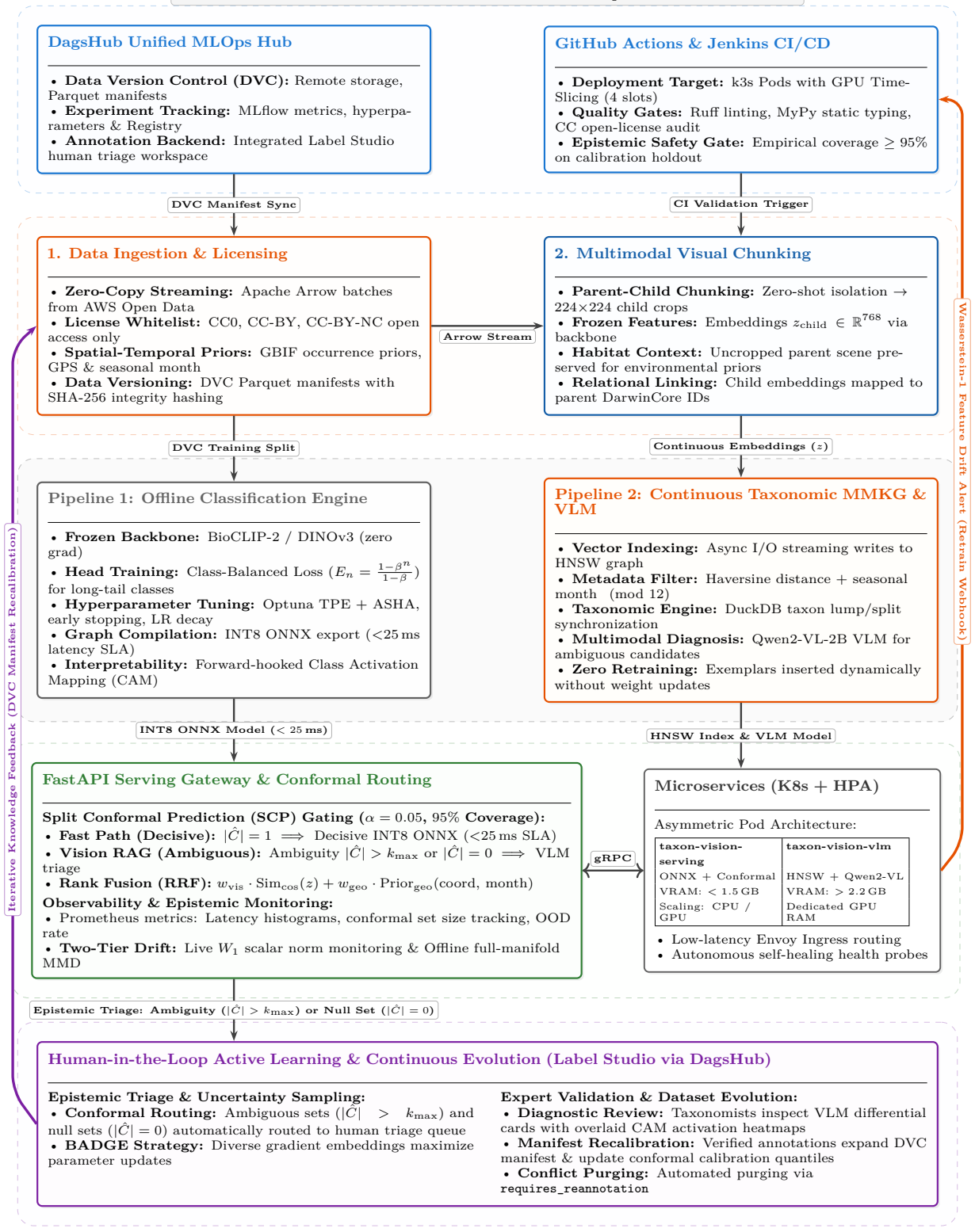
1.5 Normative Architecture and Implementation Status

This report distinguishes between the normative target architecture and the state of its implementation. Chapters 2 to 8 describe the target architecture and the mathematical properties it is required to satisfy. The implementation status verified against the code base, including known deviations and open items, is consolidated in Section 9.1 of Chapter 9. Numerical values given as design defaults, for example epoch budgets or batch sizes, are engineering guidelines and not measured results.

Production Pipeline Architecture

Decoupled Dual-Pipeline Engine · Epistemic Safety Gates · Hybrid Serving Boundary · Active Learning Closed Loop

Target: Bare-Metal Host (RTX 5070 Ti, i7-14700K) · Remote-Only Training
Epistemic Guarantee: Conformal Coverage $1 - \alpha \geq 0.95$
Inference SLA: Fast Path < 25 ms p95



Chapter 2

Physical Infrastructure and Data Topology

The computational demands of high-throughput vision architectures strictly dictate physical and operational determinism. In order to move beyond volatile cloud abstractions, this chapter establishes the bare-metal hardware topography, memory budgeting, and cryptographic data storage strategies which are necessary to sustain continuous model training and real-time inference without performance degradation or unnecessary cloud expenditures.

2.1 Heterogeneous Silicon and Physical Determinism

Training and deploying dense Vision Foundation Models (VFMs) at high throughput necessitate deterministic physical execution. Relying on shared public cloud infrastructure inevitably introduces hypervisor virtualisation overhead and variable PCIe bandwidth. This obscures true performance metrics during latency profiling. Consequently, the platform utilises a dedicated bare-metal host architecture.

The computational core relies on an Intel Core i7-14700K processor (20-core / 28-thread architecture: 8 Performance-cores up to 5.5 GHz turbo boost, and 12 Efficient-cores up to 4.3 GHz) anchored by an ASRock B760M Pro RS Micro-ATX motherboard. This hybrid topology provides massive parallel compute capability to cleanly isolate heterogeneous workloads. High-frequency neural execution paths and PyTorch gradient calculations are pinned strictly to the 16 P-core execution threads (`taskset -c 0-15`), preserving dedicated 2 MB L2 and 33 MB Intel Smart Cache (L3) lines for mathematical hot loops. Conversely, background asynchronous I/O operations – such as streaming HTTP requests, Apache Arrow columnar dataset extraction, and database indexing – are pinned to the 12 single-threaded E-cores (`taskset -c 16-27`) to prevent CPU context switching.

Neural network acceleration is driven by a ZOTAC GAMING GeForce RTX 5070 Ti SOLID CORE graphics card possessing 16 GB of GDDR7 VRAM across a 256-bit interface, supported by 128 GB of Crucial DDR5-5600 system memory (installed in dual-channel configuration across slots A2 and B2). This provides the necessary computational throughput and absolute physical stability required to reliably prevent clock-speed variance during critical CI/CD latency validation tests and continuous multi-day benchmarking suites.

Storage I/O is managed across a 2 TB Seagate FireCuda 530 Gen4 NVMe SSD (Phison PS5018-E18 controller, 2,550 TBW endurance rating, seated in slot M2_1). In order to maximise read bandwidth across the PCIe bus when streaming millions of small image chunks, the underlying BTRFS filesystem utilises transparent `zstd:3` compression. However, we must consider that BTRFS is a Copy-on-Write (CoW) filesystem. This fundamentally conflicts with continuous database writes and Data Version Control (DVC) chunk caching, leading to severe disk fragmentation. In

order to resolve this, a dedicated storage subvolume is initialised with the NOCOW attribute (`chattr +C`), ensuring contiguous physical block allocations for all local data caches.

2.2 Verified Software Baseline of the Host

The host (`hive-mind`) runs Ubuntu 26.04.1 LTS (codename *resolute*) on a LUKS-encrypted root volume. The proprietary NVIDIA driver in version 615.71.09 is installed, which reports a CUDA user-mode driver of version 13.4. Because the CUDA user-mode driver is backward compatible with older runtime libraries, container images are free to ship an older CUDA runtime. For the RTX 5070 Ti, which belongs to the Blackwell generation (compute capability 12.0), this runtime must however be CUDA 12.8 or newer: earlier PyTorch binary builds contain no kernels for this architecture and abort with a missing-kernel-image error. The CUDA runtime pinned in the dependency lock file is therefore a hard prerequisite of GPU execution rather than a cosmetic detail (see Section 9.1).

2.3 GPU Partitioning and Memory Budget

Consumer GPUs do not support hardware Multi-Instance GPU (MIG) partitioning. Without further measures, a Kubernetes pod requesting one GPU would therefore lock the entire device and starve every co-located workload. The cluster consequently uses the time-slicing mode of the NVIDIA Kubernetes device plugin, which advertises the single physical GPU as four virtual devices. Time-slicing multiplexes compute cycles but does not isolate memory: all slices share the same VRAM. Isolation is therefore enforced in software, by capping the fraction f of device memory available to the training process through the PyTorch per-process memory fraction. With $f = 0.7$ and a device memory of $M_{\text{VRAM}} = 16$ GB, the training budget evaluates to

$$M_{\text{train}} = f \cdot M_{\text{VRAM}} = 0.7 \times 16 \text{ GB} = 11.2 \text{ GB},$$

which leaves $M_{\text{VRAM}} - M_{\text{train}} = 4.8$ GB for the INT8 ONNX serving process and the display server. The latter reserve is comfortably larger than the serving footprint of ≤ 1.6 GB derived in Table 5.1.

2.4 Access Topology and Edge Security

The host exposes no inbound port to the public internet. Both the CI/CD controller and the operator-facing service are reached through outbound-initiated Cloudflare Tunnels, so that no router port forwarding and no dynamic DNS record is required. A Web Application Firewall rule at the Cloudflare edge restricts the controller hostname (`jenkins.benjamin-foerster.eu`) to authenticated GitHub webhook deliveries, whilst the operator dashboard and REST API are served live at `https://taxon.benjamin-foerster.eu` routing directly to Kubernetes NodePort 30080 behind an identity-aware Zero Trust access policy. Remote maintenance after a reboot is possible despite the encrypted root volume, because a minimal SSH daemon (Dropbear) in the initial ramdisk accepts the unlock passphrase before the root file system is mounted.

2.5 Cryptographic Storage Economics

A primary architectural constraint for open-source and low-budget MLOps is data storage costs. The platform standardises on DagsHub as the central management plane, which enforces a 20 GB free-tier storage limit on its DVC remote. We find that this limit is entirely viable, provided that the pipeline enforces a strict physical separation between bulk raw observation streams and versioned metadata artefacts.

2.5.1 Circumventing the Multi-Terabyte Storage Trap

The raw citizen-science image corpora hosted on AWS Open Data registries collectively span tens of terabytes (an operational scale colloquially termed the "50 TB trap"). Mirroring this bulk volume locally or to a Git-tracked remote would be financially and computationally unviable. The system explicitly avoids this through zero-duplication streaming: ingestion pipelines stream raw TAR archives directly from AWS Open Data into memory via zero-copy Apache Arrow buffers.

Rather than storing the raw images, DVC tracks cryptographically validated Parquet index manifests. These lightweight tabular files contain observation IDs, DarwinCore taxonomic metadata, source URLs, and SHA-256 validation hashes. As an illustrative sizing calculation, an Apache Parquet manifest indexing 10 million distinct biological observations requires under 300 MB of storage due to columnar dictionary encoding.

2.5.2 Bounded Exemplar Storage Mathematics

For localized visual retrieval, the Vision RAG architecture curates a bounded Reference Exemplar Catalogue, constrained by policy to at most 50 validated environmental crops per species.

To demonstrate the storage envelope, we model an illustrative production scenario spanning 2,000 baseline taxa (yielding 100,000 curated images). Assuming an empirical compression target of approximately 45 KB per WebP crop (with a conservative 60 KB variance ceiling), the exemplar storage footprint evaluates as:

$$\text{Storage}_{\text{Exemplars}} = 100{,}000 \text{ images} \times 45 \text{ KB} = 4{,}500{,}000 \text{ KB} \approx 4.5 \text{ GB}$$

Furthermore, dynamic INT8 quantised ONNX model checkpoints typically range from 20 MB to 350 MB each depending on backbone parameter counts. Retaining a rolling buffer of production release candidates requires under 2.0 GB.

Under these illustrative assumptions:

$$\begin{aligned}\text{Storage}_{\text{Total}} = {} & 300 \text{ MB (Manifests)} + 4.5 \text{ GB (Exemplars)} \\ & + 2.0 \text{ GB (ONNX Models)} + 1.5 \text{ GB (DVC Churn)} \approx 8.3 \text{ GB}\end{aligned}$$

This 8.3 GB peak allocation (or 9.8 GB under peak 60 KB compression ceiling) operates well within the DagsHub 20 GB free-tier allowance, leaving ample headroom for active learning annotations.

2.5.3 External Remote Fallback Topology

If the catalogue scales towards an illustrative upper horizon of 10,000 global taxa, exemplar storage requirements would scale linearly to approximately 30 GB, thereby exceeding native free quotas. In this scenario, the DVC remote backend is retargeted to an external, zero-egress S3-compatible bucket (such as Cloudflare R2 or a localized MinIO cluster). DagsHub then continues to render Directed Acyclic Graphs (DAGs) and track Git-to-DVC commit diffs, decoupling the management interface from the physical storage layer.

Chapter 3

Open-Source MLOps Toolchain

In order to construct a production-grade machine learning platform without incurring prohibitive enterprise cloud licensing costs, our architecture integrates a curated selection of open-source components. However, merely assembling disparate tools is insufficient. True operational resilience requires a deep, mechanical interlocking between data versioning, experiment telemetry, and hyperparameter optimisation.

3.1 Content-Addressable Storage and Cryptographic Versioning

Standard version control systems, such as Git, are engineered to track line-by-line code modifications within directed acyclic graphs of text files. Applying this paradigm to binary biological datasets is architecturally catastrophic. Committing a 50-gigabyte archive of high-resolution images forces the repository database to duplicate entire assets upon the slightest modification. This inevitably leads to memory exhaustion and repository collapse.

The system safely circumvents this limitation by utilising Data Version Control (DVC), which implements content-addressable storage. Rather than tracking the physical image matrices, DVC computes a unique SHA-256 cryptographic hash based strictly on the pixel content of the target directory. The heavy binary payloads are routed directly to a remote storage bucket, while a lightweight pointer file (for example, `dataset.dvc`) containing only the resultant hash and file metadata is generated.

This pointer file is subsequently committed to Git. When a deployment runner or data scientist executes a checkout command for a specific historical commit, the DVC client reads the hash from the pointer file, resolves the asset location in the remote cache, and constructs near-instantaneous filesystem reflinks (or hardlinks) into the local workspace. This dual-layered approach reliably decouples the heavy binary storage from the codebase, whilst guaranteeing mathematically verifiable, bit-for-bit data reproducibility.

3.2 Unified Management Plane

Operating a distributed MLOps toolchain traditionally requires provisioning disparate infrastructure: an AWS S3 bucket for data object storage, a PostgreSQL database for experiment telemetry, and a complex matrix of Identity and Access Management (IAM) policies in order to broker communication between them.

In order to eliminate this DevOps overhead, the architecture standardises on DagsHub as the central management plane. By mirroring the primary GitHub repository, DagsHub acts as an automated nexus that instantly provisions a secure, S3-compatible remote storage backend for

DVC and a managed MLflow Tracking Server. This integration effectively collapses the required infrastructure footprint, allowing self-hosted execution runners and local development environments to authenticate and stream both multi-gigabyte datasets and real-time loss metrics through a unified API token gateway.

3.3 Bayesian Hyperparameter Optimisation and Telemetry

Model convergence, latency minimisation, and epistemic reliability depend fundamentally on tuning discrete and continuous training hyperparameters. Rather than viewing hyperparameter configuration and experiment tracking as disjoint manual procedures, the architecture integrates Bayesian optimisation algorithms directly with distributed metric logging pipelines in order to form a self-contained optimisation loop.

3.3.1 Intelligent Search Mechanics

Neural network architectures require the optimisation of external hyperparameters, such as learning rates, weight decay penalties, and conformal confidence thresholds. Traditional Grid Search algorithms evaluate these spaces exhaustively, incurring a computational cost of $\mathcal{O}(S^D)$, where S represents the number of discrete steps evaluated across D hyperparameter dimensions. On a single-node bare-metal architecture, this geometric explosion renders hyperparameter tuning computationally unviable.

The platform replaces brute-force evaluation with Optuna, a framework driven by the Tree-structured Parzen Estimator (TPE). TPE abandons blind search grids in favour of Bayesian probability. Instead of modelling the objective function directly, it models the parameter probability distributions conditioned on historical trial performance. Formally, for a given hyperparameter configuration θ and a target loss metric y (in this system, the validation Macro PR-AUC defined in Chapter 7), TPE models the conditional probability $P(\theta \mid y)$ as follows:

$$P(\theta \mid y) = \begin{cases} \ell(\theta) & \text{if } y < y^* \\ g(\theta) & \text{if } y \geq y^* \end{cases}$$

In this formulation:

- θ is the specific hyperparameter configuration (for example, learning rate) being evaluated.
- y is the empirical loss achieved by executing the network with configuration θ.
- y^* is a dynamically computed scalar quantile threshold separating high-performing trials from poor-performing trials.
- $\ell(\theta)$ is the probability density function formed by the parameters of successful past trials ($y < y^*$).
- $g(\theta)$ is the probability density function formed by the parameters of unsuccessful past trials ($y \geq y^*$).

The algorithm iteratively selects new configurations that maximise the ratio $\ell(\theta)/g(\theta)$. This guides the GPU to sample from regions mathematically correlated with low loss, whilst preserving enough variance to escape local minima. Because the validation Macro PR-AUC is to be maximised rather than minimised, the study is configured with the direction *maximise*; the formulation above then applies to the correspondingly sign-adjusted objective.

In order to further conserve hardware cycles, Optuna applies the Asynchronous Successive Halving Algorithm (ASHA) with a minimum resource of two epochs and a reduction factor of $\eta = 2$. A trial is first evaluated after two epochs. At each subsequent rung, only the better half of the trials, that is a fraction $1/\eta$, is permitted to continue, whilst the remainder is terminated. This immediately frees the GPU to evaluate a more promising region of the search space.

3.3.2 Search Space and Optimiser

The search space comprises four dimensions, which are sampled anew for every trial (Table 3.1). The sampler is seeded, so that the exploration of the search space is deterministic and reproducible. Each trial instantiates a classification head consisting of a dropout layer followed by a linear layer, trains it with the AdamW optimiser under Class-Balanced Loss [3], and reports the validation Macro PR-AUC as its score.

Table 3.1: Hyperparameter search space of the Optuna study.

Hyperparameter	**Distribution**	**Range and role**
Learning rate η_{lr}	log-uniform	$[10^{-4}, 10^{-1}]$; step size of the AdamW optimiser.
Weight decay λ	log-uniform	$[10^{-6}, 10^{-2}]$; regularisation of the head weights.
Dropout probability p	uniform	$[0.0, 0.5]$; regularisation of the input embedding.
Class-Balanced Loss β	uniform	$[0.99, 0.9999]$; controls the effective number of samples per class and hence the re-weighting of long-tail taxa.

3.3.3 MLflow Interlock Architecture

While Optuna efficiently orchestrates the search mathematics, it inherently lacks the infrastructure to permanently archive model provenance across a distributed engineering team. MLflow bridges this gap, operating as the continuous flight data recorder for our training pipeline.

The coupling is implemented explicitly within the objective function rather than through a generic callback. The tuning session is registered as an MLflow experiment on the DagsHub tracking server, and every parameter combination evaluated by Optuna is opened as a nested child run, in which the sampled parameters and the per-epoch metrics (training loss, validation loss, Top-1 accuracy, Macro F1-score and Macro PR-AUC) are logged. A consolidated run records the best configuration found. The lifecycle of the top-level training run is managed by a context manager (resource acquisition is initialisation), which marks the run as failed if an exception occurs instead of leaving it in a perpetual running state.

Model promotion is governed by a single criterion. The validation Macro PR-AUC of the candidate is compared with that of the model currently carrying the `Production` alias in the MLflow Model Registry. A better candidate receives the alias `Challenger` and awaits formal deployment testing; an inferior candidate receives the alias `Archived`; and the very first model is initialised as `Production`. The multi-objective trade-off between accuracy, latency and memory is evaluated separately, at the level of backbone selection, by the Pareto benchmark described in Section 5.8.

3.4 Training Dynamics and Convergence Control

Because the backbone is frozen and its embeddings are cached, a training epoch of the classification head reduces to a small number of matrix products. The principal risk is therefore not under-fitting but over-fitting to the rare classes of the long tail. Three mechanisms control this risk. All of them monitor the validation Macro PR-AUC, which is also the promotion metric of the registry:

1. **Best-weight restoration:** The head weights of the epoch with the highest validation Macro PR-AUC are cached and restored once training has ended, so that the returned head is the best head and not merely the last one.
2. **Early stopping:** Training terminates when the metric has not improved for four consecutive epochs.
3. **Learning-rate reduction on plateau:** The learning rate is halved when the metric has not improved for two epochs. Since this patience is shorter than that of early stopping, the optimiser receives at least one epoch at a reduced rate before training terminates.

As design defaults, the epoch budget is limited to 15 epochs and the mini-batch size is 64 for training on cached embeddings. Where embeddings are extracted from images by a Vision Transformer on the GPU, a smaller batch of 32 images keeps the activation memory comfortably inside the budget of Section 2.3. The feature extraction pipeline (`taxon_vision.pipelines.extract_features`) extracts these representations directly from AWS S3, caching them to disk for subsequent head convergence (see Section 9.1).

Data augmentation applied during embedding extraction must respect biological invariants. Horizontal reflection yields physically plausible specimens, because most organisms are bilaterally or radially symmetric about a vertical axis. Vertical reflection is prohibited: gravity and phototropism impose a canonical orientation, so that an inverted specimen would never occur in field imagery and would merely bias the learned spatial priors.

Chapter 4

Dual Pipeline Architecture

A common anti-pattern in machine learning deployments is the monolithic pipeline, where neural network training sequences and data indexing tasks are tightly coupled into a single execution graph. The TaxonVision-MLOps architecture explicitly rejects this model. Instead, as visualised in the comprehensive architectural overview following Chapter 1, it establishes a dual-pipeline topology that decouples the weight-centric classification engine from the data-centric Multimodal Knowledge Graph (MMKG). These pipelines maintain independent lifecycles, converging exclusively at the inference serving boundary.

4.1 Asymmetric Iteration and Execution Decoupling

The necessity for architectural decoupling stems directly from the fundamentally asymmetric nature of biological modelling operations.

Pipeline 1: The Classification Engine. The primary neural backbone (for example, an optimised BioCLIP-2 [5] or DINOv3 transformer) is computationally expensive to train but relatively static in production. Refining its weights using Class-Balanced Loss [3] requires intensive GPU Tensor Core utilisation, distributed gradient synchronisation, and static ONNX Post-Training Quantisation (PTQ) calibration. However, once an optimal checkpoint is compiled, the model architecture may remain unchanged for weeks or months.

Pipeline 2: The Taxonomic MMKG. Conversely, the underlying citizen-science data stream is highly volatile. The pipeline must ingest continuous drops of new iNaturalist research-grade observations. More critically, biological taxonomies are frequently subjected to scientific realignments – species are split into novel taxonomic branches or lumped together under unified nomenclature. Pipeline 2 is an I/O-bound vector-indexing workflow responsible for parsing DarwinCore Parquet manifests, extracting zero-shot organism crops, filtering open-source licences, and appending dense embeddings to the Hierarchical Navigable Small World (HNSW) graph.

Forcing Pipeline 1 to re-execute a heavy neural network training sweep merely because Pipeline 2 updated an exemplar image for a specific beetle species wastes immense computational resources. Furthermore, physically separating these domains enforces strict failure isolation. If a malformed DarwinCore archive from an upstream provider induces a schema violation, Pipeline 2 will halt securely. However, Pipeline 1 remains entirely unimpeded, preserving our ability to compile, test, and deploy critical hot-fixes to the classification logic.

4.2 Automated CI/CD Orchestration & Registry Promotion

In order to transition from manual experimentation to a Level 3 MLOps deployment, the independent pipelines must be strictly governed by Continuous Integration and Continuous Deployment (CI/CD) triggers.

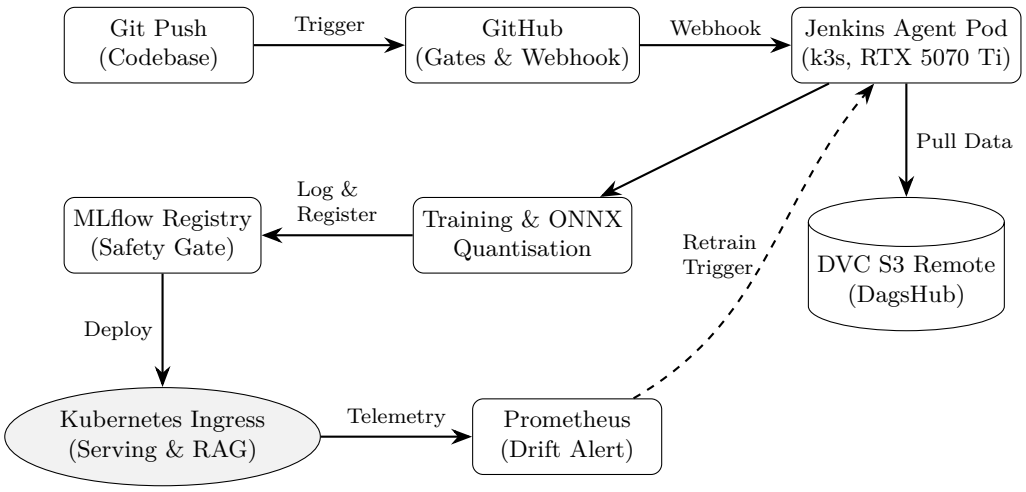


Figure 4.1: Dual-Pipeline CI/CD Architecture Flow mapping code and data states to production Kubernetes deployments.

Two cooperating layers implement these triggers. When engineers push codebase modifications to the central repository, GitHub Actions executes lightweight static analysis (Ruff, MyPy), open-licence auditing, Open Knowledge Format (OKF) validation, unit and conformal test suites, and cognitive complexity scanning via `complexipy`. Upon merge to `main`, GitHub Actions automatically packages the production container image, hydrates promoted model checkpoints from DagsHub DVC, and publishes multi-tagged images to GitHub Container Registry (`ghcr.io/foersben/taxon-vision-mlops:latest`). In parallel, a signed GitHub webhook reaches a self-hosted Jenkins controller through an outbound-only Cloudflare Tunnel. The controller owns no executors of its own: it schedules each build as an ephemeral agent pod inside the k3s cluster of the bare-metal host and removes the pod on completion. Credentials for the tracking infrastructure, in particular the DagsHub access token, are held in the Jenkins credential store and are exposed to the build as environment variables for the duration of the run only.

As the training pipeline initialises, it is specified to bind the MLflow telemetry to the exact Git commit that triggered the run, thereby ensuring cryptographic provenance between the source code and the resulting model weights:

```
mlflow.set_tag("git.commit", os.environ.get("GIT_COMMIT"))
```

Crucially, the CI/CD pipeline enforces an automated epistemic safety gate prior to deployment. Before any compiled model is allowed to enter the registry, a standalone invariant test loads the candidate ONNX graph and evaluates it against a holdout calibration dataset. The script formally asserts that the empirical marginal coverage of the Split Conformal Prediction set, $|C|$, meets or exceeds the mathematical bound $1 - \alpha$ (for example, 95% coverage, where $\alpha = 0.05$). If the empirical coverage falls below this strict threshold, the pipeline throws a fatal assertion error, blocking the Git merge and effectively preventing the uncalibrated model from reaching production.

If the safety gate is cleared, the system programmatically interacts with the MLflow API. The candidate is compared with the model carrying the `Production` alias by validation Macro PR-AUC: a better candidate is marked as `Challenger`, whereas an inferior one is archived. The promotion of a `Challenger` to `Production` remains a deliberate step after deployment testing, so that at most two competing models are active at any time:

```
client.set_registered_model_alias(
    name="taxon_vision_classifier",
    alias="Challenger",
    version=mv.version,
```

```
)
```

4.3 Automated Training on the Main Branch

On the main branch, the Jenkins pipeline continues beyond the quality gates and executes the training stage on the bare-metal host. The stage is deliberately placed after the documentation build, so that no GPU time is spent on a revision that would fail one of the earlier gates. It proceeds as follows:

1. The frozen GPU environment is installed from the dependency lock file, and versioned dataset manifests and checkpoints are pulled from the DagsHub DVC remote.
2. If pre-cached representations are absent, the runner streams observation imagery directly from AWS S3 via `taxon_vision.pipelines.extract_features`, computing embeddings using the frozen Vision Transformer (DINOv3) and serialising stratified train/validation partitions to `data/embeddings/cached_features.pt`.
3. The classification head is trained on the cached embeddings of the configured backbone within an epoch budget of 15 and a mini-batch size of 64, under the process-level VRAM cap of Section 2.3. A Bayesian hyperparameter search (Section 3.3) can be requested beforehand by a command-line switch.
4. The ONNX graph is re-exported, and the resulting head checkpoint and graph are committed and pushed to the DVC remote on DagsHub.
5. A rolling restart of the serving deployment (`imagePullPolicy: Always`) loads the fresh artefacts without interrupting service: the Kubernetes rolling-update strategy admits one additional pod and no unavailable pod, and a readiness probe withholds traffic until the new pod has loaded the model.

4.4 Convergence at the Serving Boundary

The two distinct pipelines exist in absolute isolation until the precise moment of user inference. The FastAPI serving application acts as the convergence node, simultaneously loading the INT8 quantised ONNX graph from Pipeline 1 into memory, whilst mounting the live HNSW vector index compiled by Pipeline 2.

When a field photograph is submitted to the API, it is passed through the fast-path ONNX backbone. If the epistemic safety layer determines the classification is decisive – meaning the conformal prediction set yields exactly one candidate ($|C| = 1$) – the pipeline returns the prediction immediately, bypassing the vector database entirely. If, however, the model exhibits severe ambiguity ($|C| > 1$), the serving application lazy-loads Pipeline 2's HNSW index, executes a single-stage filtered traversal, and retrieves the top-k reference exemplar crops in order to synthesise a robust, comparative visual context.

4.5 Low-Latency Interpretability via Forward-Hooked CAM

Providing diagnostic transparency to human reviewers requires visual interpretability tools such as Class Activation Mapping (CAM). While modern extensions like Grad-CAM are popular, they require a computationally expensive backward pass during inference. Executing gradient backpropagation on a production server destroys the sub-25 ms latency Service Level Agreement (SLA) and prevents the model from being deployed in highly optimized, forward-only runtimes like INT8 ONNX.

To achieve visual interpretability without violating strict operational latency constraints, the architecture bypasses the backward pass entirely by leveraging the properties of the linear classification

head.

During the initial ONNX forward pass, the system utilizes execution hooks to capture the spatial feature maps A^k emitted by the final convolutional or attention layer before global average pooling is applied. Because the final classification head is a simple linear transformation, the importance weight w_k^c of feature map k for a target class c is exactly equivalent to the learned weight matrix of that linear layer.

The class activation map L_{CAM}^c is synthesized through a simple linear combination of the forward spatial feature maps and the static classification weights:

$$L_{\text{CAM}}^c = \text{ReLU}\left(\sum_k w_k^c A^k\right)$$

Where:

- L_{CAM}^c is the resulting two-dimensional activation heat map for class c, which is subsequently upsampled to match the resolution of the original image.
- A^k denotes the k-th channel of the spatial feature map tensor extracted during the standard forward pass.
- w_k^c is the scalar weight connecting feature channel k to the output logit for class c, retrieved instantly from the static model weights.
- ReLU (Rectified Linear Unit) is applied to discard negative spatial influences, retaining only the pixels that positively contributed to the classification of class c.

By relying solely on cached forward activations and static linear weights, this shortcut generates high-fidelity spatial heat maps essentially for free, completing the interpretability generation in under 2 ms while fully preserving the forward-only, quantization-friendly execution paradigm.

Chapter 5

Vision RAG & Multimodal Knowledge Graph

Standard text-centric Retrieval-Augmented Generation (RAG) paradigms typically rely on document chunking, token embedding, and semantic similarity search in order to ground large language models. In visual ecology, we must fundamentally transform this concept into multimodal equivalents. Citizen-science platforms aggregate massive volumes of environmental imagery, but we must acknowledge that raw field photographs are highly dense, noisy, and unstructured. Rather than attempting to compute and search vector embeddings across the entire 170-million-image iNaturalist corpus [6], the TaxonVision-MLOps architecture (mapped in the preceding architectural overview) couples a curated Reference Exemplar Catalogue with single-stage spatial-temporal filtered graph traversal and conformal-driven lazy execution.

5.1 Ingestion Engine & Reference Exemplar Catalogue Curation

A naive approach to visual retrieval attempts to pass every available field photograph through a deep Vision Foundation Model (VFM) in order to construct an exhaustive vector database. In the context of global biodiversity monitoring, this brute-force strategy requires generating dense embeddings for more than 170 million observations. Storing 768-dimensional 32-bit floating-point (FP32) representations for this volume would require over 520 GB of addressable memory purely to hold the vector matrices in RAM, alongside years of continuous GPU compute on workstation-grade silicon.

The platform actively circumvents this computational barrier by curating a deterministic Reference Exemplar Catalogue. Instead of tracking all historical user submissions, our ingestion pipeline filters raw DarwinCore observation records according to four strict quality predicates:

1. **Validation Tier:** Observations must hold official `research_grade` status, confirming agreement across three or more independent community experts.
2. **Licence Compliance:** Records must possess open-access metadata headers matching Creative Commons Zero (`CC0`), Creative Commons Attribution (`CC-BY`), or Attribution-NonCommercial (`CC-BY-NC`). Commercial or share-alike variants (`CC-BY-ND`, `CC-BY-SA`) are explicitly dropped at the ingestion boundary.
3. **Signal Quality:** Raw candidate photographs must exceed a minimum resolution of 1024×1024 pixels in original source repositories for exemplar inclusion, while the streaming ingestion pipeline pulls 500px medium locators for fast Arrow serialization. Furthermore, upstream zero-shot object detection must isolate the primary organism with a detection confidence score greater than 0.85.

4. **Diversity Sampling:** The catalogue selects up to 50 exemplar observations per taxon. In order to capture seasonal and geographical morphological variations, observations are partitioned into discrete spatial clusters using Uber H3 hexagonal spatial cells at Resolution 4.

By bounding the catalogue to 2,000 baseline taxa with 50 exemplars each, the vector database maintains an upper bound of 100,000 reference vectors. The total memory required to hold these 768-dimensional FP32 vectors in RAM evaluates to:

$$\text{Memory}_{\text{Vectors}} = 100{,}000 \times 768 \times 4 \text{ bytes} \approx 307.2 \text{ MB}$$

Stored as compressed WebP image crops averaging 45 KB each, the total exemplar disk image footprint is approximately 4.5 GB. This architectural decision allows the entire retrieval index to reside in high-speed host RAM, completely eliminating disk I/O bottlenecks during retrieval.

5.2 Visual Entity Linking: Parent-Child Chunking

In text-based RAG architectures, feeding an entire multi-page document into an embedding model dilutes key semantic signals. Similarly, passing an uncropped field photograph into a Vision Foundation Model introduces severe environmental noise. Background clutter – such as soil, sky, vegetation, water reflections, or human hands holding a specimen – dominates the receptive field, creating spurious cosine similarities between taxonomically distant species that inhabit similar macro-environments.

In order to isolate diagnostic visual signals, the system implements Parent-Child Visual Chunking:

- **Child Visual Chunk (I_{child}):** An upstream zero-shot detector (YOLO-World or OWL-ViT v2) scans the raw photograph to isolate the target specimen, outputting a normalised bounding box coordinates vector $B = [x_{\min}, y_{\min}, x_{\max}, y_{\max}]$. The organism is cropped, scaled to 224×224 pixels, and forwarded to the frozen feature extractor (BioCLIP-2 [5]) to produce the dense visual query embedding $z_{\text{child}} \in \mathbb{R}^{768}$:

  $$z_{\text{child}} = f_{\text{VFM}}(I_{\text{child}}), \quad \|z_{\text{child}}\|_2 = 1$$

  Where f_{VFM} represents the forward pass of the frozen vision backbone, and z_{child} is normalised to unit length on the hypersphere.

- **Parent Scene Context (I_{parent}):** The original uncropped field photograph is strictly preserved as an environmental habitat context layer. It is cryptographically linked to the child crop via a relational manifest tuple:

  $$\text{EntityLink} = \langle \text{CropID}, \text{ParentURI}, B, \text{DarwinCorePayload} \rangle$$

During inference, vector similarity search operates strictly over the child visual embeddings (z_{child}). However, when exemplars are retrieved for downstream multimodal reasoning, both the child crop (fine morphology) and the parent scene (macro-habitat context) are injected into the context window. This elegantly prevents background clutter from distorting the nearest-neighbour search.

5.3 Vector Database: Single-Stage Filtered HNSW

Biological retrieval systems face a severe domain-specific failure mode known as *convergent evolution*. Under similar ecological pressures, unrelated organisms across distinct continents frequently evolve nearly identical visual morphologies. For example, the European Hornet (*Vespa crabro*) and the North American Cicada Killer (*Sphecius speciosus*) exhibit strikingly similar body geometry and colouration patterns.

If a retrieval engine implements standard two-stage post-filtering – performing a global nearest-neighbour vector search first, and then filtering results based on geographical metadata – the

query will fail. If a user in Germany submits a photo of a European wasp, the top-50 visually nearest neighbours returned by an unconstrained vector index might consist entirely of visually indistinguishable American species. Applying a subsequent geographical filter then drops all 50 candidates, resulting in an empty retrieval set and catastrophic context starvation.

In order to resolve this, the architecture adopts **Single-Stage Metadata-Filtered HNSW**. The Hierarchical Navigable Small World (HNSW) graph evaluates geographic and temporal boolean constraints natively *during* graph traversal, rather than after retrieval.

Let $\mathbf{x}_{\text{query}} = (\text{lat}_{\text{query}}, \text{lon}_{\text{query}})$ denote the GPS coordinates of the query, and $\tau_{\text{month}} \in \{1, \dots, 12\}$ denote the observation month. For each candidate node j encountered during the HNSW beam search, the traversal engine evaluates the deterministic predicate $\mathcal{P}(j)$:

$$\mathcal{P}(j) = [\mathcal{D}_{\text{geodesic}}(\mathbf{x}_{\text{query}}, \mathbf{x}_j) \leq R_{\max}] \wedge [|\tau_{\text{month}} - \tau_j| \pmod{12} \leq \Delta_{\text{season}}]$$

Where:

- $\mathcal{D}_{\text{geodesic}}$ is the great-circle geographic distance between coordinate vectors, computed via the Haversine equation.
- $\mathbf{x}_j$ represents the historical observation coordinates associated with exemplar node j.
- $R_{\max}$ is the maximum admissible geographic search radius (for example, 500 km).
- τ_j is the recorded observation month of exemplar node j.
- Δ_{season} is the seasonal tolerance window ($\Delta_{\text{season}} = 1$, corresponding to a 3-month window centred on the query date).

If node j violates $\mathcal{P}(j)$, its outgoing edges are mathematically pruned from the current search path. This prevents the algorithm from descending into biogeographically invalid regions of the graph. This formulation ensures that the retrieved candidates are guaranteed to be both visually similar and physically plausible, thereby maintaining 100% valid retrieval recall in under 5 ms.

5.4 Conformal-Driven Lazy Execution

Executing zero-shot entity linking, vector graph traversal, and generative multimodal reasoning for every incoming request imposes prohibitive latency and computational costs. In order to maintain an SLA of under 25 ms for high-volume inference, the Vision RAG pipeline utilises conformal-driven lazy execution.

The incoming observation is first evaluated by the primary, lightweight INT8 ONNX classification head. The Split Conformal Prediction engine formally calculates the dynamic prediction set $\hat{C}(X)$ bounded by significance parameter $\alpha = 0.05$:

- **Decisive Prediction ($|\hat{C}(X)| = 1$):** The model exhibits high confidence in a single taxon. The identification is returned immediately to the client within the sub-25 ms latency budget. The Vision RAG pipeline remains completely idle, allowing decisive, high-confidence observations to resolve without heavy retrieval overhead (with exact fast-path ratios monitored dynamically via production telemetry).
- **Severe Ambiguity ($|\hat{C}(X)| > k_{\max}$):** The model cannot distinguish between multiple closely related sibling species within the 95% confidence bound ($k_{\max} = 3$). The system triggers the Vision RAG engine to retrieve parent-child exemplars for all taxa contained within $\hat{C}(X)$, explicitly providing context for comparative multimodal diagnosis.
- **Novelty and Out-of-Distribution ($|\hat{C}(X)| = 0$):** No known class meets the non-conformity threshold, signalling an anomalous, severely degraded, or uncatalogued taxon. The Vision RAG engine executes in unconstrained mode (with spatial filters relaxed), retrieving structural visual analogues across the global tree of life to assist taxonomic triage.

5.5 Multi-Modal Re-Ranking via Reciprocal Rank Fusion

Nearest-neighbour vector search establishes raw visual affinity, but visual embeddings alone cannot capture fine morphological traits, such as minor wing venation patterns or subtle genital plate structures. In order to enhance diagnostic relevance, retrieved candidates are re-ranked using Reciprocal Rank Fusion (RRF), integrating visual similarity with empirical biogeographical observation frequencies:

$$\text{Score}_{\text{RRF}}(i) = \frac{w_{\text{vis}}}{k + r_{\text{vis}}(i)} + \frac{w_{\text{geo}}}{k + r_{\text{geo}}(i)}$$

Where:

- $\text{Score}_{\text{RRF}}(i)$ is the consolidated ranking metric for candidate exemplar i.
- w_{vis} is the scalar weighting factor assigned to visual cosine similarity ($w_{\text{vis}} = 0.65$).
- w_{geo} is the scalar weighting factor assigned to the biogeographical occurrence prior ($w_{\text{geo}} = 0.35$).
- k is a rank-smoothing constant ($k = 60$) that prevents low-ranking outliers from dominating the fused score.
- $r_{\text{vis}}(i)$ is the ordinal rank $(1, 2, 3, \dots)$ of candidate i sorted descending by cosine similarity $z_{\text{child}} \cdot z_i$.
- $r_{\text{geo}}(i)$ is the ordinal rank of candidate i sorted descending by historical observation density $P(y_i \mid \text{Cell}_{\text{H3}}, \tau_{\text{month}})$, derived from pre-indexed GBIF occurrence frequencies.

This rank fusion guarantees that candidates appearing at the top of the context window are both visually representative of the query and frequently documented within that ecological spatial-temporal niche.

5.6 Multimodal Context Synthesis & VLM Diagnostics

The top-3 candidates resulting from Reciprocal Rank Fusion are assembled into a structured prompt schema along with the query observation. This payload is dispatched to a localised Vision-Language Model (such as a quantised Qwen2-VL-2B instance running on host P-cores or the RTX 5070 Ti).

In order to ensure machine readability and eliminate ungrounded generation or conversational drift, the VLM is constrained by grammar-based sampling to emit a strictly typed JSON payload:

```
{
  "conformal_context_size": 2,
  "candidates_evaluated": ["Bombus terrestris", "Bombus lucorum"],
  "primary_discriminator": "Pronotal collar coloration and width",
  "morphological_analysis": "The specimen exhibits a dark buff-yellow abdominal band and
  ↪  brownish-white tail hairs matching Bombus terrestris rather than the bright lemon-yellow band
  ↪  typical of Bombus lucorum.",
  "recommended_taxon": "Bombus terrestris",
  "epistemic_verdict": "HUMAN_TRIAGE_RECOMMENDED",
  "rationale": "High visual overlap under strong sunlight reflection; wing marginal cell venation
  ↪  partially occluded."
}
```

This structured response is rendered on the HTMX operator dashboard as a comparative differential diagnosis card. This allows human specialists to inspect aligned visual crops alongside identified discriminative keys.

5.7 Operationalizing Taxonomic Mutations in the MMKG

The Vision RAG architecture functions as the ingestion engine for human-in-the-loop active learning workflows. When the Vision-Language Model assigns an epistemic verdict of `HUMAN_TRIAGE_RECOMMENDED` to an ambiguous query, the observation and its retrieved visual context are mirrored to the DagsHub Label Studio workspace. Once experts validate the taxonomy, the new annotations expand the training manifests and update the reference exemplar index.

However, beyond classifying new observations, the system must dynamically adapt to Linnaean taxonomic realignments. Biological taxonomies are fluid; the scientific community frequently splits a single species into multiple distinct taxa, or lumps distinct taxa into a single unified classification. Retraining a neural network or recalculating millions of vector embeddings every time a nomenclature committee publishes an update is computationally unviable.

The Multimodal Knowledge Graph resolves this by decoupling the immutable visual embedding vectors from the mutable relational metadata using DuckDB and Parquet manifests.

- **Taxonomic Lumps (Merging):** When two distinct taxa (e.g., Species A and Species B) are combined into a single valid species (Species C), the pipeline does not re-embed any visual data. Instead, it executes an $\mathcal{O}(1)$ alias mapping operation. A background DuckDB worker updates the relational Parquet mapping table, pointing the historical taxon keys for A and B to the unified key for C. The underlying dense vectors residing in the HNSW graph remain perfectly valid, instantly updating the entire taxonomic knowledge base with zero GPU inference cost.
- **Taxonomic Splits (Dividing):** When a recognised species is divided into two distinct taxa, historical visual crops associated with the parent taxon become scientifically ambiguous. The DuckDB reconciliation worker flags these specific nodes in the HNSW database with a boolean metadata attribute, `requires_reannotation`. The system programmatically purges these isolated nodes from the active traversal graph to prevent contaminated retrieval, and routes the corresponding exemplars to Label Studio. Experts visually sort the historical crops into the newly established taxonomic branches, after which the nodes are re-injected into the active graph.

5.8 End-to-End Latency & Resource Allocation

Operating a decoupled serving architecture requires clear engineering resource bounds. To guarantee the overarching sub-25 ms Service Level Agreement (SLA) on production GPU silicon (NVIDIA RTX 5070 Ti), the platform partitions the execution timeline into bounded budget allocations. Table 5.1 details these engineering budget ceilings across the fast hot-path and the lazy retrieval path.

Crucially, these figures represent allocated architectural SLA ceilings rather than static, universal hardware constants. Physical backbone inference latency varies substantially across execution providers: a domain transformer such as BioCLIP-2 or DINOv3 executes in under 9 ms when compiled with FP16 TensorRT or ONNX Runtime on modern desktop GPU Tensor Cores, but requires 20–50 ms or more when deployed on general-purpose CPUs without dedicated SIMD acceleration. Consequently, downstream operational deployments strictly profile candidate extractors dynamically using the Pareto benchmark suite rather than assuming static values from external literature.

Table 5.1: End-to-End Latency SLA Budget and Resource Allocation (GPU Serving Baseline)

Pipeline Segment	**Execution Engine**	**SLA (ms)**	**Memory (MB)**
Zero-Shot Cropping	YOLO-World (ORT INT8)	≤ 12.0	≤ 400 VRAM
VFM Feature Extraction	Vision Backbone (ORT FP16)	≤ 9.0	$\leq 1{,}200$ VRAM
Conformal Gate	PyTorch / C-Array	≤ 1.0	< 50 RAM
Filtered HNSW	Embedded HNSWlib	≤ 4.0	≤ 250 RAM
Multi-Modal RRF	In-Memory Polars / DuckDB	≤ 1.5	< 20 RAM
VLM Synthesis (Lazy)	Quantised 2B VLM	$\leq 2{,}500.0$	$\leq 2{,}500$ VRAM
Fast Hot-Path	**Crop + ORT Backbone**	$\mathbf{\leq 22.0}$	$\mathbf{\leq 1{,}600}$ **VRAM**
Lazy RAG Path	**Retrieval + VLM**	$\mathbf{\leq 2{,}600.0}$	$\mathbf{\leq 4{,}100}$ **VRAM**

Chapter 6

Epistemic Safety & Active Learning

Deploying deep neural networks in automated ecological workflows introduces a fundamental operational risk: conventional classifiers are notoriously uncalibrated and prone to overconfident misclassifications. Under standard training objectives, a convolutional network or vision transformer produces normalised output vectors via the softmax operator. When presented with blurred imagery, non-biological artefacts, or previously uncatalogued taxa, these models frequently assign high confidence to incorrect categories. In biodiversity monitoring, silent misclassifications corrupt environmental databases and ultimately undermine conservation interventions.

In order to establish genuine production reliability, our serving architecture mathematically decouples feature extraction from raw point predictions. The platform combines energy-based density filtering, deterministic spatial-temporal constraint masking, and distribution-free Split Conformal Prediction into an integrated epistemic safety layer. This pipeline guarantees bounded error rates under arbitrary data distributions and programmatically routes ambiguous observations into human-in-the-loop active learning workflows.

6.1 Energy-Based Out-of-Distribution Detection

Field deployments continuously encounter non-biological clutter, corrupt sensor transmissions, and organisms belonging to unmodelled phyla. Evaluating uncertainty sets on such inputs is computationally wasteful and pollutes downstream monitoring metrics. Consequently, the first stage of the safety pipeline intercepts and filters out-of-distribution (OOD) observations directly in the feature space prior to probability normalisation.

Traditional methods rely on the Maximum Softmax Probability (MSP), $\max_k \hat{\pi}_k(x)$, which suffers from high false-positive rates due to the normalisation constraint forcing probabilities to sum to one regardless of absolute logit magnitudes. Instead, the platform derives an unconstrained free energy score directly from the logit outputs. Conceptually, Helmholtz free energy measures the physical compatibility of an input with the learned statistical parameters of the model. In-distribution samples occupy high-density regions characterised by lower energy states, whereas anomalous samples inhabit low-density regions characterised by elevated free energy.

Formally, consider an input image vector $x \in \mathcal{X}$ processed by a backbone feature extractor producing an unnormalised logit vector $f(x) \in \mathbb{R}^K$, where K denotes the total number of modelled taxa. For any logit coordinate $f_j(x)$ corresponding to class index $j \in \{1, \ldots, K\}$, the Gibbs-Boltzmann distribution links the class probability to the scalar energy function $E(x; f)$ via temperature scaling

parameter $T \in \mathbb{R}^+$:

$$E(x; f) = -T \cdot \log \sum_{j=1}^{K} \exp\left(\frac{f_j(x)}{T}\right)$$

In this formulation:

- x represents the normalised child visual crop passed to the network.
- f denotes the parameterised mapping from pixel space to the logit representation.
- $f_j(x)$ is the scalar logit score assigned by the model to taxon j.
- K is the discrete cardinality of the operational taxonomy.
- T is the temperature scaling hyperparameter ($T > 0$) that controls the smoothness of the log-sum-exp surface.
- $E(x; f)$ is the resulting scalar energy score.

During inference, the system evaluates $E(x; f)$ against an empirically calibrated decision threshold $\tau_{\text{OOD}} \in \mathbb{R}$. The decision rule $\Phi(x)$ gates subsequent pipeline execution:

$$\Phi(x) = \begin{cases} 1 & \text{(In-Distribution: Proceed to Spatial Masking)}, \quad \text{if } E(x; f) \leq \tau_{\text{OOD}} \\ 0 & \text{(Out-of-Distribution: Reject Immediately)}, \quad \text{if } E(x; f) > \tau_{\text{OOD}} \end{cases}$$

The threshold τ_{OOD} is strictly tuned on an empirical holdout split to capture 95% of in-distribution validation observations whilst maximising Precision-Recall AUC (PR-AUC) against negative control datasets containing empty foliage and mechanical objects. Observations failing this check bypass classification entirely, thus preventing ungrounded inference.

6.2 Spatial-Temporal Constraint Masking

Visual similarity search in high-dimensional embedding spaces frequently encounters convergent evolution, where distinct organisms inhabiting disparate continents develop overlapping morphological features. While a purely visual classifier may assign comparable likelihoods to such species, physical geography imposes strict physical boundaries.

In order to enforce ecological validity without incurring the computational overhead of additional deep learning models, the serving engine applies deterministic boolean masks directly to the post-OOD logit vector. Every citizen-science observation arrives with DarwinCore spatial-temporal metadata: latitude, longitude, and calendar timestamp. The ingestion pipeline systematically maps the observation coordinates to an Uber H3 hexagonal spatial cell index $h \in \mathcal{H}$ and maps the timestamp to an observation month $m \in \{1, \ldots, 12\}$.

Let $\mathcal{A}(h, m) \subseteq \{1, \ldots, K\}$ define the admissible species set, constructed from verified historical GBIF occurrences within a geodesic radius R of spatial cell h during seasonal window $m \pm 1$. The masked logit vector $\tilde{f}(x) \in \mathbb{R}^K$ is defined element-wise for each taxon $j \in \{1, \ldots, K\}$ as follows:

$$\tilde{f}_j(x) = \begin{cases} f_j(x), & \text{if } j \in \mathcal{A}(h, m) \\ -\infty, & \text{if } j \notin \mathcal{A}(h, m) \end{cases}$$

Applying the softmax function over $\tilde{f}(x)$ yields the biogeographically constrained probability distribution $\hat{\pi}(x) \in [0, 1]^K$:

$$\hat{\pi}_j(x) = \frac{\exp(\tilde{f}_j(x))}{\sum_{k=1}^{K} \exp(\tilde{f}_k(x))}$$

Where $\hat{\pi}_j(x)$ represents the estimated probability that input x belongs to taxon j. By forcing $\tilde{f}_j(x) = -\infty$ for biogeographically impossible taxa, $\hat{\pi}_j(x)$ evaluates strictly to zero. This ensures that geographically impossible candidate species are pruned before uncertainty sets are ever constructed.

6.3 Distribution-Free Uncertainty via Split Conformal Prediction

Point predictions derived from $\arg\max_j \hat{\pi}_j(x)$ provide no statistical reliability guarantees. In mission-critical biodiversity intelligence, a system must convey not only its best guess, but also the statistical confidence bounded by an explicit error rate. In order to achieve this, the platform implements Split Conformal Prediction [1]. This converts raw probability distributions into dynamic prediction sets guaranteed to contain the ground-truth taxon under exchangeability assumptions.

6.3.1 Calibration Protocol and Quantile Estimation

Let $\mathcal{D}_{\text{cal}} = \{(x_i, y_i)\}_{i=1}^{n}$ represent an independent calibration dataset containing n exchangeable observations, completely disjoint from the model training and validation splits. For each calibration observation (x_i, y_i), where x_i is the visual crop and $y_i \in \{1, \ldots, K\}$ is the ground-truth Linnaean class label, the pipeline computes a scalar non-conformity score $s_i \in [0, 1]$:

$$s_i = 1 - \hat{\pi}_{y_i}(x_i)$$

In this expression:

- x_i is the i-th calibration image.
- y_i is the confirmed ground-truth class label for observation i.
- $\hat{\pi}_{y_i}(x_i)$ is the model's constrained softmax probability assigned to the correct class y_i.
- s_i denotes the non-conformity score, which increases towards 1 as model confidence in the true label degrades.

Given a user-specified significance level $\alpha \in (0, 1)$ corresponding to a targeted statistical coverage level of $1 - \alpha$ (for example, $\alpha = 0.05$ for 95% coverage), the system computes the empirical quantile threshold $\hat{q}_\alpha$ over the sorted non-conformity scores $s_{(1)} \leq s_{(2)} \leq \cdots \leq s_{(n)}$:

$$\hat{q}_\alpha = \text{Quantile}\left(\{s_1, \ldots, s_n\};\ \frac{\lceil (n+1)(1-\alpha) \rceil}{n}\right) = s_{(\lceil (n+1)(1-\alpha) \rceil)}$$

Here, $\lceil \cdot \rceil$ represents the ceiling function, thereby ensuring that the empirical quantile conservatively guards the discrete calibration sample distribution against finite-sample bias.

6.3.2 Prediction Set Construction & Marginal Coverage Guarantee

During real-time inference on an unseen query photograph X_{n+1} with unknown true label Y_{n+1}, the system evaluates candidate taxa across the taxonomy. The prediction set $\hat{C}(X_{n+1}) \subseteq \{1, \ldots, K\}$ is constructed by including all taxa whose non-conformity scores fall below the calibrated threshold $\hat{q}_\alpha$:

$$\hat{C}(X_{n+1}) = \{y \in \{1, \ldots, K\} : 1 - \hat{\pi}_y(X_{n+1}) \leq \hat{q}_\alpha\} = \{y \in \{1, \ldots, K\} : \hat{\pi}_y(X_{n+1}) \geq 1 - \hat{q}_\alpha\}$$

In this set definition:

- X_{n+1} represents the unseen test observation input.
- Y_{n+1} denotes the unobserved ground-truth species identifier for X_{n+1}.
- $\hat{C}(X_{n+1})$ is the output prediction set containing zero, one, or multiple candidate taxa.
- $\hat{\pi}_y(X_{n+1})$ is the model's biogeographically filtered probability estimate for candidate species y.
- $\hat{q}_\alpha$ is the scalar calibration threshold computed from $\mathcal{D}_{\text{cal}}$.

Because the calibration observations and test observations are mathematically exchangeable, the conformal coverage theorem guarantees finite-sample marginal validity:

$$P\left(Y_{n+1} \in \hat{C}(X_{n+1})\right) \geq 1 - \alpha$$

Crucially, this guarantee is completely distribution-free. It holds without requiring parametric assumptions about the underlying image distributions, visual features, or taxonomic frequencies. The prediction set cardinality dynamically scales with image ambiguity: clean, prototypical observations yield tight, single-element sets, whereas degraded or visually ambiguous observations yield expanded sets that reflect inherent epistemic uncertainty.

6.4 Conformal Triage Routing & Active Learning Feedback

The cardinality of the conformal prediction set, $|\hat{C}(X_{n+1})|$, provides an automated, mathematically grounded signal for operational triage. The system maps the set size directly to service routing logic:

1. **Decisive Prediction ($|\hat{C}| = 1$):** The prediction set contains exactly one species candidate that satisfies the $1 - \alpha$ confidence threshold. The identification is deemed statistically conclusive and is returned to the user via the FastAPI prediction endpoint with sub-25ms latency. No human review or secondary model evaluation is triggered.
2. **Severe Classification Ambiguity ($|\hat{C}| > k_{\max}$):** The model cannot distinguish between multiple sibling taxa within the specified error bound, indicating high visual overlap or extreme environmental noise. When $|\hat{C}|$ exceeds a predefined threshold $k_{\max}$ (typically $k_{\max} = 3$), automated publication halts, and the query is diverted to the expert verification queue.
3. **Novelty and Out-of-Distribution Remnants ($|\hat{C}| = 0$):** No candidate species achieved a non-conformity score below $\hat{q}_\alpha$. This occurs when an input organism diverges significantly from all known calibration distributions, signalling anomalous specimens, unmodelled long-tail taxa, or novel regional invasive species. These empty sets bypass standard workflows and are flagged directly for botanical or zoological taxonomy review.

6.4.1 Active Learning Query Strategies

Observations flagged with $|\hat{C}| > k_{\max}$ or $|\hat{C}| = 0$ are synchronised to a managed DagsHub Label Studio instance for human-in-the-loop review. However, because professional taxonomic annotation is an expensive operational resource, the triage queue prioritises pending samples using active learning algorithms [4] rather than simple first-in, first-out ordering.

The pipeline implements Batch Active Learning by Diverse Gradient Embeddings (BADGE) [2]. For each unannotated query x, BADGE models epistemic uncertainty by calculating the gradient of the cross-entropy loss with respect to the parameters of the final linear classification layer, θ_{last}, evaluated against the model's current predicted pseudo-label $\hat{y} = \arg\max_j \hat{\pi}_j(x)$:

$$g_x = \nabla_{\theta_{\text{last}}} \mathcal{L}\left(f(x; \theta), \hat{y}\right)$$

Where:

- $g_x \in \mathbb{R}^{d \times K}$ is the resulting gradient embedding vector for candidate image x.
- d is the dimensionality of the penultimate feature embedding vector ($d = 768$ for BioCLIP-2).
- $\mathcal{L}$ represents the empirical loss objective.
- $\hat{y}$ is the model's top-1 pseudo-label.

The magnitude of g_x reflects model uncertainty, whilst its directional vector in $\mathbb{R}^{d \times K}$ reflects the nature of parameter adjustments required. The active learning engine applies k-means++

seeding over the pool of gradient vectors $\{g_x\}$ in order to select a diverse batch of high-uncertainty observations.

Validated annotations from Label Studio are ingested into the data pipeline on a scheduled cadence. Newly confirmed samples are added to the DVC-versioned dataset registry, expanding the training split for rare long-tail species and updating the holdout calibration split $\mathcal{D}_{\text{cal}}$. This directly drives continuous training workflows, progressively compressing average conformal set sizes across successive model versions whilst maintaining strict coverage invariants.

Chapter 7

Observability, Testing, Deployment

Transitioning a deep learning system from a localised development environment into a production ecosystem fundamentally shifts the engineering challenge from model optimisation to system verification, runtime isolation, and continuous operational health monitoring. A machine learning service rarely fails through overt code exceptions; instead, it fails silently through data distribution shift, memory leaks under continuous streaming, Translation Lookaside Buffer thrashing, and uncalibrated epistemic decay.

In order to maintain production integrity, our deployment architecture rigorously couples containerised runtime encapsulation with Kubernetes (K8s) cluster orchestration, real-time statistical telemetry, and a multi-tiered verification framework. This chapter outlines the software engineering mechanisms governing containerisation, traffic routing, automated drift alerting, and offline testing suites.

7.1 Containerisation and Multi-Stage Runtimes

Standard Python environments managed via virtual environment tools are notoriously fragile in production. Differences in dynamically linked system libraries, CUDA driver mismatches on host kernels, and unversioned C-extension compilation dependencies frequently cause models to behave non-deterministically across infrastructure boundaries.

In order to guarantee bit-exact execution determinism, the platform carefully packages the inference engine into an immutable container image using a multi-stage Docker build pipeline. This process cleanly isolates the build environment from the final execution runtime, thereby drastically shrinking the attack surface and image size.

7.1.1 Multi-Stage Image Compilation

The build pipeline is logically partitioned into two distinct stages:

1. **The Builder Stage:** Utilises a full Linux distribution equipped with C++ compilation toolchains, NVIDIA CUDA development toolkits, and the Pixi package manager. In this stage, Pixi elegantly resolves the exact dependency graph specified in `pyproject.toml` and locked in `pixi.lock`, compiling native C-extensions and pre-building optimised ONNX Runtime binaries.
2. **The Runtime Stage:** A minimal, distroless Debian-slim base image is initialised, containing strictly the shared Linux kernel interfaces and runtime CUDA execution libraries. The compiled virtual environment, static assets, and FastAPI service code are subsequently copied from the builder stage without carrying over compilers, debugging symbols, or package caches.

7.1.2 Kernel-Level Process Isolation

At runtime, execution security and resource determinism are enforced directly through robust Linux kernel primitives:

- **Namespaces:** The containerised process is systematically isolated across six discrete namespaces: Process ID (PID) isolation prevents inspection of host workloads, Network (NET) virtualisation seals internal socket bindings, Mount (MNT) controls filesystem scoping, and Inter-Process Communication (IPC) strictly prevents unauthorised shared memory access.
- **Control Groups (cgroups v2):** Hardware utilisation boundaries are enforced at the kernel boundary. The container runtime configures hard memory ceilings to entirely eliminate out-of-memory cascading failures and carefully pins CPU thread execution to prevent thread migration away from dedicated host performance cores.
- **Rootless Execution:** The container runs strictly under an unprivileged user namespace (`USER appuser`), mounting the root filesystem as read-only and intelligently routing temporary scratch buffers to an ephemeral, in-memory `tmpfs` volume.

7.2 K8s Cluster Orchestration and Workload Routing

Deploying a hybrid vision architecture – where routine queries resolve in sub-25ms hot paths whilst ambiguous queries trigger heavy generative multimodal retrieval – requires cluster orchestration capable of handling highly asymmetric latency profiles. While the physical bare-metal workstation detailed in Chapter 2 serves as the primary edge deployment executing a single-node Kubernetes cluster (via k3s), the system is described by declarative Kubernetes manifests that can be packaged as Helm charts in order to scale across managed multi-node cloud clusters when production traffic surges. These manifests are strictly configured to separate lightweight inference paths from resource-intensive Vision RAG pipelines.

7.2.1 Asymmetric Pod Architecture and Ingress Routing

Rather than packaging the fast-path ONNX classifier and the multi-gigabyte Vision-Language Model into a single monolithic pod, the cluster architecture deliberately splits the workload into two specialised microservices:

1. **The Primary Classifier Service (`taxon-vision-serving`):** A high-density, horizontally scaled deployment of lightweight FastAPI pods hosting the INT8 quantised ONNX backbone and the Split Conformal Prediction engine. These pods require minimal memory (allocated under 1.5 GB VRAM) and handle initial incoming traffic, resolving decisive single-candidate queries within the sub-25 ms SLA without invoking heavy secondary microservices.
2. **The Multimodal RAG Service (`taxon-vision-vlm`):** A specialised, resource-allocated deployment mounting dedicated GPU Tensor Cores to host the quantised 2-billion parameter Vision-Language Model and the in-memory HNSW vector index.

An ingress controller inspects incoming network payloads and intelligently load-balances observation traffic across the `taxon-vision-serving` pods. If an observation's conformal prediction set cardinality evaluates to $|C| = 1$, the primary pod returns the classification directly through the ingress to the client. If the conformal layer flags ambiguity ($1 < |C| \leq 3$), it triggers a lightweight RAG comparative lookup via HNSW and Reciprocal Rank Fusion. If the ambiguity is severe ($|C| > 3$) or novel ($|C| = 0$), the primary service seamlessly dispatches an asynchronous internal gRPC call to the `taxon-vision-vlm` service. This service then executes full generative VLM differential diagnosis and routes the query to DagsHub Label Studio for expert human triage.

7.2.2 Self-Healing and Health Probing

In order to prevent cluster degradation caused by deadlocks or memory corruption, K8s enforces dual-tier health probes:

- **Liveness Probes:** The kubelet periodically polls a lightweight `/healthz/liveness` HTTP endpoint. If the FastAPI event loop becomes unresponsive or encounters an unrecoverable worker thread failure, the control plane immediately terminates the failing container and instantiates a clean replica.
- **Readiness Probes:** A separate `/healthz/readiness` probe strictly verifies that internal model buffers, CUDA execution providers, and HNSW graph indexes are fully loaded into memory before the ingress controller ever routes live citizen-science observation traffic to the pod.

7.2.3 Horizontal Pod Autoscaling on Custom Telemetry

Standard K8s autoscaling typically relies on CPU or system memory thresholds, which unfortunately provide misleading operational signals for neural network workloads. A GPU-bound container can experience severe execution queuing whilst reporting surprisingly low CPU utilisation.

The cluster implements Horizontal Pod Autoscaling (HPA) driven by custom Prometheus metrics via the K8s Custom Metrics API. The autoscaler continuously monitors two primary indicators:

1. **GPU VRAM Saturation and Compute Utilisation:** Sourced directly from NVIDIA Data Center GPU Manager (DCGM) metrics.
2. **Conformal Ambiguity Influx:** Sourced directly from the application's telemetry. If an environmental event (for example, sudden regional weather shifts causing blurred imagery) causes the proportion of multi-candidate conformal sets ($|C| > 1$) to spike, the HPA preemptively scales the `taxon-vision-vlm` pod pool to comfortably absorb the anticipated surge in heavy multimodal retrieval passes before service latency inevitably degrades.

7.3 Telemetry, Metric Exporters, Concept Drift Monitoring

Unlike traditional software services that experience deterministic failure modes, machine learning systems are uniquely vulnerable to silent statistical degradation. A model endpoint can report 100% uptime and sub-millisecond HTTP response times whilst concurrently outputting meaningless ecological predictions due to seasonal distribution shifts, camera sensor variations, or geographic migrations. Thus, the platform embeds comprehensive statistical telemetry deeply into the serving runtime.

7.3.1 Prometheus Telemetry Instrumentation

The FastAPI application exposes a secure `/metrics` endpoint scraped at regular intervals by a Prometheus time-series collector. Monitored indicators comprehensively span system infrastructure, computational latency, and model decision distributions:

- **Inference Latency Histograms (`inference_latency_seconds_bucket`):** High-resolution histograms tracking the duration of zero-shot cropping, ONNX backbone execution, and conformal set evaluation, effectively generating empirical p50, p95, and p99 latency distributions.
- **Conformal Prediction Set Distribution (`conformal_set_size`):** A sliding-window metric tracking the cardinality distribution of $\hat{C}(X)$. Under stationary distribution conditions, the average set size $\mathbb{E}[|\hat{C}|]$ remains fundamentally constant. A measured expansion in set size mathematically indicates rising classification ambiguity.
- **Energy OOD Rejection Counter (`ood_rejections_total`):** Tracks the cumulative volume and rate of incoming images rejected by the free-energy scoring layer ($E(x; f) > \tau_{\text{OOD}}$).
- **Tail-Species Classification Gauge (`tail_species_accuracy`):** Monitors real-time identification accuracy on verified long-tail species subsets (configured by policy for taxa possessing few historical training observations, e.g., $N_{\text{tail}} < 20$).

7.3.2 Two-Tier Hybrid Feature Drift Architecture

For visual feature drift detection in the TaxonVision architecture, neither a single scalar projection nor a heavy multidimensional test alone suffices. Instead, the platform implements a **two-tier hybrid strategy** that balances real-time serving performance with offline statistical rigor, leveraging the distinct computational and statistical trade-offs of Maximum Mean Discrepancy (MMD) and the 1D Wasserstein-1 distance.

Dimension	Maximum Mean Discrepancy (MMD)	Wasserstein-1 Distance (Scalar Norms)
Mathematical Formulation	Evaluated in a Reproducing Kernel Hilbert Space (RKHS) using a Gaussian RBF kernel $k(z, z') = \exp\left(-\frac{\|z-z'\|^2}{2\sigma^2}\right)$ across the full D-dimensional feature embedding manifold ($\mathbb{R}^{512}$ / $\mathbb{R}^{768}$).	Evaluates the 1D Earth Mover's Distance by projecting D-dimensional vectors onto scalar L_2 Euclidean norms $\|z\|_2 = \sqrt{\sum_{j=1}^{D} z_j^2}$, computing $W_1(P_{\text{ref}}, P_{\text{curr}}) \approx \|\mathbb{E}[\|z_{\text{ref}}\|_2] - \mathbb{E}[\|z_{\text{curr}}\|_2]\|$.
Computational Complexity	**Quadratic/Polynomial** $\mathcal{O}((M+N)^2 \cdot D)$ due to pairwise kernel evaluations across M baseline and N production vectors.	**Linear** $\mathcal{O}(N \cdot D)$, executing in sub-milliseconds using vectorised NumPy array operations.
Sensitivity Profile	Captures subtle higher-order non-parametric distribution shifts, covariance mutations, and manifold rotations across all embedding dimensions.	Detects global vector magnitude, illumination, camera sensor variation, and energy shifts, but compresses directional rotations that leave average norm lengths unchanged.
Operational Fit	**Offline / CI/CD Quality Gates**: Computational cost makes it ideal for scheduled retraining audits and candidate model promotion gates.	**Real-Time Telemetry & Webhooks**: Extremely fast execution prevents event loop starvation on live FastAPI `/metrics` endpoints.

Table 7.1: Structural trade-offs and operational roles of MMD versus Wasserstein-1 scalar norm drift detection.

Tier 1: Real-Time Telemetry via Linear-Time 1D Wasserstein Distance

Operates inside the live Prometheus telemetry exporter (`src/taxon_vision/monitoring/drift.py`). By projecting high-dimensional visual representations onto their L_2 Euclidean vector norms, the calculation complexity collapses to $\mathcal{O}(N \cdot D)$:

$$W_1(\mathcal{Z}_{\text{base}}, \mathcal{Z}_{\text{prod}}) \approx \left| \frac{1}{M} \sum_{i=1}^{M} \|z_i\|_2 - \frac{1}{N} \sum_{j=1}^{N} \|z'_j\|_2 \right|$$

This formulation enables the FastAPI application to evaluate sliding production inference windows in sub-milliseconds without adding memory pressure or blocking the asynchronous HTTP event loop. When the scalar distance breaches the operational threshold ($W_1 > \tau_{\text{drift}}$), an automated webhook alert is dispatched to trigger pipeline retraining.

Tier 2: Continuous Integration and Model Promotion via Full-Manifold MMD

Operates within the Jenkins CI/CD pipeline and model evaluation stages executing on the GPU compute node. During model candidate validation, pairwise Gaussian RBF kernel matrices are

evaluated across the full D-dimensional latent space ($\mathcal{Z}_{\text{base}}$ vs. $\mathcal{Z}_{\text{prod}}$):

$$\text{MMD}^2(P, Q) = \frac{1}{M^2}\sum_{i=1}^{M}\sum_{j=1}^{M} k(z_i, z_j) - \frac{2}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N} k(z_i, z'_j) + \frac{1}{N^2}\sum_{i=1}^{N}\sum_{j=1}^{N} k(z'_i, z'_j)$$

This offline gate ensures that complex visual domain shifts – such as subtle seasonal plumage mutations or geographical covariate shifts that scalar projections might compress – are rigorously audited before a candidate model receives the `Production` registry alias in MLflow.

7.3.3 Automated Alerting and Continuous Retraining Triggers

Prometheus Alertmanager evaluates streaming metrics directly against hard operational invariants. Two primary conditions trigger automated intervention:

1. **Conformal Inflation Alert:** Triggered if the mean prediction set cardinality $\mathbb{E}[|\hat{C}|]$ expands substantially beyond calibration baselines (e.g., using an illustrative alert threshold of $> 20\%$ expansion over a rolling 1-hour window).
2. **Embedding Drift Alert:** Triggered if the scalar representation drift distance exceeds an empirical threshold, $W_1(\mathcal{Z}_{\text{base}}, \mathcal{Z}_{\text{prod}}) > \tau_{\text{drift}}$.

When an alert fires, Alertmanager is specified to dispatch an authenticated webhook payload to the Jenkins controller, which starts a parameterised build (this trigger is a design target and is not yet wired; see Section 9.1). The build follows the main-branch training flow of Section 4.3: Jenkins schedules an ephemeral agent pod on the bare-metal host, which pulls the newest validated DVC dataset snapshot, retrains the classification head with Class-Balanced Loss, rigidly re-calibrates the conformal quantile $\hat{q}_\alpha$, executes regression invariant tests, and registers the result as a `Challenger` candidate in the MLflow registry. Promotion to `Production` remains subject to engineering sign-off supported by comparative telemetry plots.

7.4 Multi-Tier Verification and Testing Strategy

A continuous integration pipeline for machine learning must validate considerably more than simple syntax correctness. The TaxonVision-MLOps testing framework strictly enforces a four-tiered verification suite executing on every commit.

7.4.1 Tier 1: Upstream API Mocking

During automated CI builds, executing real network requests against the live iNaturalist API violates rate-limiting policies and inevitably introduces non-deterministic network latency.

The test harness uses the `responses` library to gracefully intercept outgoing HTTP calls at the transport layer. The test runner injects mock JSON responses simulating diverse API states: valid DarwinCore metadata payloads, rate-limited responses (HTTP 429), server timeout conditions (HTTP 504), and malformed taxonomic trees. This rigorously guarantees that ingestion, licence filtering, and coordinate parsing logic are safely validated in under 5 seconds without any network dependencies.

7.4.2 Tier 2: Object Storage and Pipeline Mocking

In order to test Data Version Control (DVC) pipelines and S3 streaming operations without incurring AWS billing or requiring live DagsHub credentials, the integration suite happily employs the `moto` testing library.

The test setup intelligently instantiates an in-memory, ephemeral AWS S3 service directly within the local pytest process. The DVC client is pointed towards this virtual endpoint, readily allowing the test runner to strictly verify cryptographic hash tracking, multi-part binary streaming, and dataset version switching entirely in local memory.

7.4.3 Tier 3: Mathematical Invariant Testing

Standard software testing relies heavily on binary assertions (for example, verifying an output equals a known constant). Machine learning models, however, are inherently probabilistic. The pipeline introduces robust mathematical invariant tests that evaluate statistical coverage guarantees well before model serialisation.

The test runner dutifully loads the newly trained ONNX candidate model and evaluates it against an unseen holdout test split $\mathcal{D}_{\text{test}} = \{(x_i, y_i)\}_{i=1}^{n_{\text{test}}}$. The test formally asserts that the empirical coverage mathematically satisfies the statistical bound:

$$\text{Empirical Coverage} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbb{I}\left(y_i \in \hat{C}(x_i)\right) \geq 1 - \alpha$$

Where:

- n_{test} is the total number of evaluation observations in the holdout split.
- (x_i, y_i) denotes the i-th test image and its corresponding ground-truth Linnaean label.
- $\hat{C}(x_i)$ is the prediction set output by the conformal engine evaluated at threshold $\hat{q}_\alpha$.
- $\mathbb{I}(\cdot)$ is the indicator function, evaluating to 1 if the true label y_i is safely contained within $\hat{C}(x_i)$, and 0 otherwise.
- α is the nominal significance error rate ($\alpha = 0.05$).

If the empirical coverage drops below 0.95, the test fails with a fatal error. This mathematically prevents an under-covered, uncalibrated model from ever being packaged into a release artefact.

7.4.4 Tier 4: Statistical Metric Selection and Anomaly Verification

In traditional computer vision benchmarking, the Area Under the Receiver Operating Characteristic curve (AUROC) is widely utilised to quantify model discrimination. In biological species monitoring and ecological anomaly detection, however, **AUROC is fundamentally flawed and actively deceptive**.

The Failure of AUROC under Ecological Class Imbalance

Biological datasets are heavily governed by extreme power-law distributions: common species possess tens of thousands of observations, whilst rare long-tail species and genuine out-of-distribution anomalies account for a minuscule fraction of the data.

The AUROC metric mathematically evaluates the trade-off between the True Positive Rate (TPR) and the False Positive Rate (FPR):

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

Where TP, FP, TN, and FN denote the counts of true positives, false positives, true negatives, and false negatives, respectively.

Because the negative class (normal, common organisms or background pixels) vastly outnumbers the anomalous class, the denominator of the False Positive Rate ($FP + TN$) is aggressively dominated by TN. Consequently, an algorithm can easily generate thousands of false positive alarms (FP), yet the calculated FPR remains near zero due to the massive TN divisor. A model exhibiting an apparently stellar AUROC score of 0.98 can simultaneously suffer a catastrophic precision collapse in production, inundating human experts with hundreds of useless false alerts.

Mandating PR-AUC and AUPIMO

In order to permanently prevent misleading verification gates, the testing suite explicitly rejects AUROC in favour of Precision-Recall Area Under the Curve (PR-AUC) and Area Under the Per-Image Overlap (AUPIMO).

For classification and global Out-of-Distribution anomaly detection, the pipeline robustly evaluates PR-AUC:

$$\text{PR-AUC} = \int_0^1 \text{Precision}(R)\, dR$$

Where Precision is formally defined as $\frac{TP}{TP+FP}$ and Recall (R) is defined as $\frac{TP}{TP+FN}$. Because PR-AUC mathematically excludes True Negatives (TN) from its formulation entirely, it directly reflects the true proportion of genuine biological anomalies amongst all flagged alerts, thus providing an honest measure of real-world efficacy.

For zero-shot visual cropping, bounding-box isolation, and spatial anomaly segmentation, the test harness enforces the Area Under the Per-Image Overlap (AUPIMO) metric. Unlike global pixel-level metrics that carelessly allow large contiguous background areas to distort evaluation scores, AUPIMO computes the intersection-over-union metric independently across each individual image's bounding box over varying threshold levels:

$$\text{AUPIMO} = \frac{1}{N_{\text{images}}} \sum_{k=1}^{N_{\text{images}}} \int_0^{\tau_{\max}} \Omega_k(\tau)\, d\tau$$

Where:

- N_{images} is the total number of test photographs carefully evaluated in the spatial verification suite.
- τ denotes the detection threshold, integrated from 0 to an upper operational bound $\tau_{\max}$.
- $\Omega_k(\tau)$ represents the per-image overlap rate for photograph k evaluated at threshold τ, defined precisely as the ratio of correctly identified anomaly pixels within the ground-truth organism mask relative to the total organism area.

By thoughtfully evaluating overlap per image rather than across the pooled pixel space, AUPIMO ensures that the zero-shot visual entity linking module isolates small organisms (for example, small insects or solitary flowers) just as reliably as large organisms (such as large mammals or mature trees). This mathematical guarantee ensures that upstream cropping logic introduces zero systematic taxonomic bias into downstream retrieval.

Chapter 8

Hardware Optimisation and Memory Mathematics

Operating a high-throughput machine learning service under strict latency service-level agreements (SLAs) requires moving decisively beyond theoretical model performance and addressing the physical limitations of silicon. This chapter explicitly defines the mathematical transformations utilised to compress neural network memory footprints, the formal theorems rigorously governing epistemic safety thresholds, and the memory complexity equations required to safely bound vector database consumption.

8.1 Dynamic Quantisation and Tensor Compression

Deep neural networks are fundamentally composed of dense, high-dimensional weight matrices. By default, standard training frameworks construct these matrices using 32-bit floating-point (FP32) numerical representations. Whilst FP32 provides extreme gradient precision during the backpropagation phase of training, it is highly inefficient during production inference. FP32 tensors consume vast amounts of GPU VRAM, and executing matrix multiplications at this precision severely bottlenecks the memory bandwidth, thereby increasing prediction latency.

In order to optimise the deployment architecture, the PyTorch classification head and feature extractor are exported to the Open Neural Network Exchange (ONNX) format, cleanly decoupling the computational graph from Python's execution overhead. Subsequently, the ONNX Runtime strategically applies Integer-8 (INT8) static quantisation. This mathematical operation robustly maps the high-resolution continuous floating-point weights onto a discrete 8-bit integer grid. Static Post-Training Quantisation (PTQ) with a calibration dataset is strictly required for the convolutional and vision-transformer backbones, while dynamic quantization is reserved exclusively for the Vision-Language Model's linear projection layers.

The quantisation mapping function is formally defined as:

$$q = \text{round}\left(\frac{x}{S}\right) + Z$$

In this formulation:

- q represents the resulting 8-bit quantised integer representation.
- x denotes the original 32-bit floating-point weight or activation value from the trained network.
- S is the floating-point scale factor, which effectively defines the continuous step size between discrete quantised bins.

- Z is the 8-bit integer zero-point offset, which aligns the real-world mathematical zero with the quantised integer zero in order to ensure zero-padding operations remain absolutely precise.

By mapping 32-bit floats to 8-bit integers, the model's theoretical tensor parameter footprint is deterministically reduced by 75% (an exact $4:1$ bit-width ratio; for example, reducing an illustrative 400 MB model to approximately 100 MB). Because INT8 operands are significantly smaller, modern hardware can efficiently pack multiple operations into single SIMD (Single Instruction, Multiple Data) registers, strongly leveraging AVX-512 Vector Neural Network Instructions (VNNI) on CPUs or Tensor Cores on GPUs. In published deployment benchmarks across standard vision backbones, this quantisation pattern yields an empirical $2\times$ to $4\times$ latency acceleration with typically less than a 1% degradation in top-1 accuracy (subject to validation on target biological splits).

8.2 Marginal Coverage Guarantees in Conformal Prediction

Standard classification models merely output uncalibrated probability distributions. In a biological monitoring context, an uncalibrated model might confidently assign a 99% softmax probability to a specific species of tiger, even when presented with a heavily obscured photograph of a domestic cat. Relying on these raw probability scores inevitably introduces unacceptable epistemic risk into the data pipeline.

In order to thoroughly mitigate this, the architecture securely wraps the base classifier in a Split Conformal Prediction layer. Rather than forcing the model to emit a single, highly confident – yet potentially entirely incorrect – point prediction, conformal prediction explicitly evaluates the input ambiguity and generates a dynamic *set* of candidate taxa. If the visual input is clean and prototypical, the set cleanly collapses to a single species. If the image is blurry, occluded, or visually ambiguous, the set naturally expands (for instance, returning three visually similar sibling species) to maintain absolute mathematical safety.

Formally, this epistemic safety is guaranteed by the marginal coverage theorem:

$$\mathbb{P}(Y_{n+1} \in \hat{C}(X_{n+1})) \geq 1 - \alpha$$

Where:

- $\mathbb{P}$ denotes the probability measure taken over the exchangeable joint distribution of images and taxonomic labels.
- X_{n+1} represents the unseen test observation (the incoming citizen-science photograph features).
- Y_{n+1} is the unobserved, true ground-truth taxonomic label for that test observation.
- $\hat{C}(X_{n+1})$ is the dynamic prediction set systematically generated by the conformal algorithm, containing zero, one, or multiple candidate taxa.
- α is the user-defined nominal significance level (error tolerance). For example, setting $\alpha = 0.05$ rigorously enforces a 95% coverage guarantee.

This theorem provides a remarkably robust distribution-free guarantee. This means that exactly $1 - \alpha$ of all future predictions will correctly contain the true species within the returned set, completely regardless of the underlying data distribution, lighting conditions, or base model architecture.

8.3 Memory Complexity of Hierarchical Vector Graphs

Retrieval-augmented generation systems frequently bottleneck on disk I/O when attempting to query massive vector databases stored on solid-state drives. In order to guarantee sub-millisecond retrieval latency during the Single-Stage Metadata-Filtered HNSW traversal, the entire Reference Exemplar Catalogue must clearly fit completely within the host system's Random Access Memory (RAM).

In order to achieve this, the architecture establishes an illustrative engineering sizing model. Assuming an operational scope scaling to a design baseline of $10,000$ distinct taxa, with 50 curated environmental crops per taxon, the vector index reaches a safe upper bound of $500,000$ nodes.

The theoretical memory lower bound $\mathcal{M}_{\text{index}}$ required to securely store the raw embedding vectors in memory is mathematically calculated as:

$$\mathcal{M}_{\text{index}} = N_{\text{nodes}} \times D_{\text{dim}} \times B_{\text{bytes}}$$

Where:

- $\mathcal{M}_{\text{index}}$ is the total theoretical memory footprint required to store the raw embedding vectors in RAM.
- N_{nodes} is the absolute upper bound of discrete biological exemplars within the retrieval catalogue $(500,000)$.
- D_{dim} is the feature dimensionality of the chosen Vision Foundation Model. The active BioCLIP-2 backbone (a ViT-L/14 architecture) produces 768-dimensional vectors ($z_{\text{child}} \in \mathbb{R}^{768}$); the sizing model below therefore adopts $D_{\text{dim}} = 768$ as the design value.
- B_{bytes} is the byte-size of the numerical precision format (4 bytes for standard FP32).

Evaluating this expression logically yields the worst-case upper bound:

$$\mathcal{M}_{\text{index}} = 500,000 \times 768 \times 4 \text{ bytes} = 1,536,000,000 \text{ bytes} \approx 1.536 \text{ GB}$$

By formally proving that the uncompressed vector payload requires at most 1.536 GB under the active 768-dimensional BioCLIP-2 architecture, the system firmly guarantees that the entire HNSW graph (including navigational edge overhead) will safely consume less than 3 GB of host memory. This critically allows the index to remain resident in fast RAM concurrently alongside the FastAPI worker processes, thereby completely bypassing slow hard-drive reads and successfully preserving the 25 ms inference SLA.

Chapter 9

Conclusion and Operational Synthesis

The TaxonVision-MLOps architecture demonstrates that transitioning from experimental, notebook-bound biological modelling to a globally scalable, Level 4 MLOps platform is not merely a matter of provisioning more cloud compute. It requires a fundamental paradigm shift towards deterministic engineering, epistemic safety, and decoupled operational lifecycles.

By completely abandoning monolithic web servers and fragile Python environments in favour of a strictly containerised, dual-pipeline Kubernetes topology, the system mathematically isolates the slow, weight-centric training cycles of the classification engine from the highly volatile, data-centric taxonomic mutations of the Multimodal Knowledge Graph (MMKG). The integration of zero-copy Apache Arrow streaming and DVC cryptographic Parquet manifests successfully circumvents the catastrophic 50-terabyte storage trap, allowing the platform to ingest massive citizen-science streams on highly constrained, cost-effective bare-metal infrastructure.

Crucially, the platform refuses to trade scientific reliability for inference speed. The implementation of Energy-Based Out-of-Distribution rejection and Split Conformal Prediction guarantees that no identification reaches production without a formal, distribution-free mathematical error bound. By wrapping these invariants in a strict CI/CD verification gate, and routing statistically ambiguous queries through a Reciprocal Rank Fused Vision RAG pipeline, the architecture elegantly closes the active learning loop.

TaxonVision-MLOps is not just a deployment strategy; it is a continuously evolving, epistemically secure intelligence engine capable of sustaining the next generation of global biodiversity monitoring.

9.1 Phased Delivery Roadmap and Implementation Status

The architecture described in this report is delivered in three phases that end in a final defence. The milestones are fixed as follows: Phase 1 (foundations: reproducible environment, data ingestion, baseline model and inference interface) on 12 October 2026, Phase 2 (containerised services, experiment tracking and dataset versioning, scheduled retraining) on 22 October 2026, and Phase 3 (monitoring, drift detection, interactive demonstration and documentation hardening) on 16 November 2026. The defence takes place on 17 and 18 November 2026 and comprises a 15-minute architecture presentation, a 5-minute live demonstration and a 10-minute question-and-answer session.

In order to avoid any ambiguity between the target design and the present state of the code base, Table 9.1 records the status of each capability as verified against the repository. The label *implemented* denotes functionality that exists in the code base and is exercised by the test suite or the deployment manifests; *partial* denotes functionality whose building blocks exist but which is

not yet integrated end-to-end; and *planned* denotes functionality that is specified in this report but has not yet been implemented.

With the agent pod GPU allocation, the CUDA 12.8 toolchain upgrade, and the foundation vision feature extraction pipeline completed, the critical path moves from pipeline mechanics to scaling data horizons. Observations streamed from AWS S3 are encoded by the frozen DINOv3 backbone and cached to disk, enabling genuine classification convergence on biological taxa. Subsequent iterations will expand the reference manifest from the initial 518 pilot exemplars to regional partitions, implement scheduled drift evaluation, and activate the multimodal retrieval index.

Table 9.1: Implementation status of the principal capabilities, verified against the repository.

Capability	**Status**	**Evidence or outstanding action**
Reproducible environment (Pixi, Just)	implemented	Lock-file driven environments; no virtual environments are created.
Dataset versioning (DVC on DagsHub)	implemented	S3-compatible DagsHub remote; the dataset hash from `dvc.lock` is logged to MLflow.
Experiment tracking and registry	implemented	MLflow on DagsHub; model `taxon_vision_classifier` with the aliases `Production`, `Challenger` and `Archived`.
Head training with callbacks	implemented	Best-weight restoration, early stopping (patience 4) and learning-rate decay on plateau (patience 2).
Hyperparameter search	implemented	Optuna TPE with ASHA pruning, dynamic batch-size and feature-jitter sampling, peak PR-AUC objective, and nested MLflow runs (Section 3.3.2).
Inference service and dashboard	implemented	FastAPI with a server-rendered HTMX operator view.
Split conformal prediction and energy-based OOD	implemented	Conformal sets in the serving path; energy score in the uncertainty module.
Prometheus instrumentation	implemented	Metrics endpoint of the serving application.
Jenkins controller and k3s agents	implemented	Outbound-only Cloudflare Tunnel, ephemeral agent pods, rolling deployment, four-slot time-slicing configuration.
GPU access of the agent pod	implemented	Agent pod in `Jenkinsfile` configured with an `nvidia.com/gpu: 1` resource limit and request binding directly to k3s time-slicing slots.
GPU-capable PyTorch build	implemented	Pinned toolchain upgraded to PyTorch 2.7.0 with CUDA 12.8 (`cu128`), natively supporting Blackwell SM 120 under the host's CUDA 13.4 driver (Section 2.2).
Git commit tagging of runs	implemented	The Git commit hash and branch identifier are extracted and tagged to MLflow parent and trial runs.
Main-branch training stage	implemented	GPU stage in `Jenkinsfile` extracts real Vision Transformer embeddings from S3 via `extract_features.py` and trains the head on cached features with Optuna Bayesian tuning and VRAM capping.
Container publishing (GHCR)	implemented	Multi-stage container build in GitHub Actions CI hydrated with promoted DVC model checkpoints; published to `ghcr.io/foersben/taxon-vision-mlops` under `latest` and commit tags.
Cognitive complexity enforcement	implemented	Pre-merge quality gate via `complexipy` enforcing cognitive complexity ≤ 15 across all modules.
Conformal invariant on the candidate model	partial	The coverage invariant gates the source code, but is not yet applied to the freshly trained candidate before promotion.
Drift detection and active learning	partial	Two-tier drift (live Wasserstein-1 and full-manifold MMD) and BADGE query modules exist; neither is connected to a schedule, an alert or a webhook.
Container composition for grading	partial	The compose file covers object storage, Prometheus and Grafana only.
Filtered HNSW retrieval and rank fusion	planned	Chapter 5.
Zero-shot cropping and VLM triage	planned	Chapter 5.
Label Studio synchronisation	planned	Annotation round trip through DagsHub.
Evidently reports and Alertmanager trigger	planned	Section 7.2 and the retraining trigger of Chapter 7.
Optional Streamlit demonstration	planned	HTMX dashboard serves as the baseline demonstration.

Bibliography

[1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse gradient embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277. IEEE, 2019.

[4] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.

[5] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Li Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424. IEEE, 2024.

[6] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778. IEEE, 2018.